



云计算 (第三版)

CLOUD COMPUTING Third Edition

第 3 章

Amazon云计算AWS

目录

3.1 基础存储架构Dynamo

3.2 弹性计算云EC2

3.3 简单存储服务S3

3.4 非关系型数据库服务SimpleDB和DynamoDB

3.5 关系数据库服务RDS

3.6 简单队列服务SQS

3.7 内容推送服务CloudFront

3.8 其他Amazon云计算服务

3.9 AWS应用实例

3.10 小结



亚马逊 Amazon

在电子商务中积累的大量基础性设施和各类先进技术，很早地进入了云计算领域，并在提供计算、存储等服务方面处于领先地位。

Amazon开发并提供了一系列云计算服务，它们共同构成了

Amazon Web Service (AWS)



提供的服务主要包括

- 弹性计算云EC2
- 简单存储服务S3
- 简单数据库服务Simple DB
- 简单队列服务SQS
- 弹性MapReduce服务
- 内容推送服务CloudFront
- 电子商务服务DevPay
- 灵活支付服务FPS



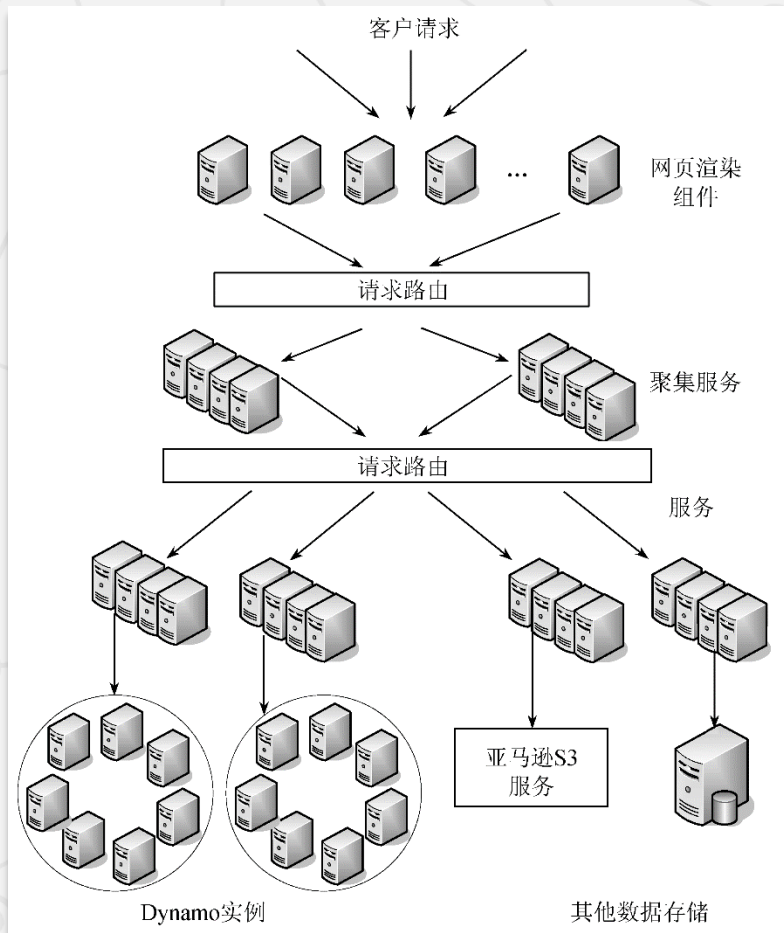
3.1 基础存储架构Dynamo

▶ 3.1.1 Dynamo概况

3.1.2 Dynamo架构的主要技术

3.1 基础存储架构Dynamo

• Dynamo概况



面向服务的Amazon平台基本架构

为了保证其稳定性，Amazon的系统采用完全的分布式、去中心化的架构

作为底层存储架构的Dynamo也同样采用了无中心的模式

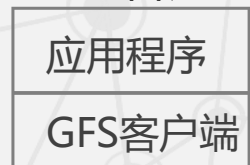
Dynamo只支持简单的键/值 (key/value) 方式的数据存储，不支持复杂的查询

Dynamo中存储的是数据值的原始形式，即按位存储，并不解析数据的具体内容

2.1 Google文件系统GFS

● GFS的系统架构

Client客户端

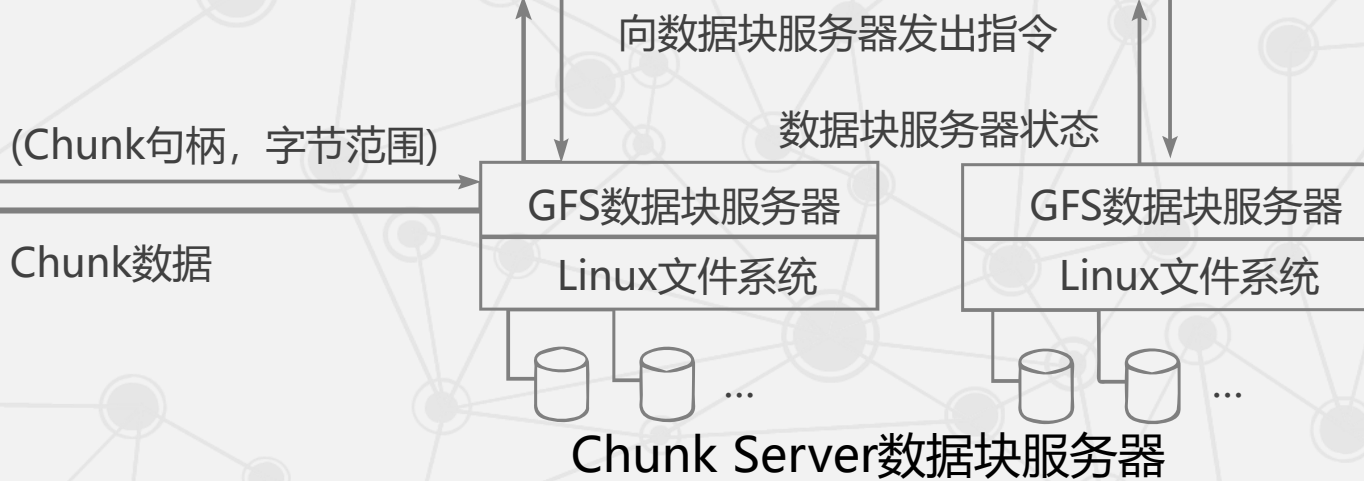


Master主服务器



标注:

➡ 数据信息
➡ 控制信息



3.1 基础存储架构Dynamo

3.1.1 Dynamo概况

► 3.1.2 Dynamo架构的主要技术

3.1 基础存储架构Dynamo

• Dynamo需要解决的主要问题及解决方案

Dynamo在设计时被定位为一个基于分布式存储架构的，高可靠、高可用且具有良好的容错性的系统。下图列举了Dynamo设计时面临的主要问题及所采取的解决方案。

问 题		采取的相关技术
数据均衡分布		
数据备份		
数据冲突处理		
成员资格及错误检测		
容错机制	临时故障处理	
	永久故障处理	

3.1 基础存储架构Dynamo

• Dynamo需要解决的主要问题及解决方案

Dynamo在设计时被定位为一个基于[分布式存储架构](#)的，高可靠、高可用且具有良好的容错性的系统。下图列举了Dynamo设计时面临的主要问题及所采取的解决方案。

问 题	采取的相关技术
数据均衡分布	改进的一致性哈希算法
数据备份	参数可调的弱quorum机制
数据冲突处理	向量时钟 (Vector Clock)
成员资格及错误检测	基于Gossip协议的成员资格和错误检测
临时故障处理	Hinted handoff (数据回传机制)
永久故障处理	Merkle哈希树

3.1 基础存储架构Dynamo

• Dynamo的存储节点

Dynamo中的存储节点呈无中心的环状分布。

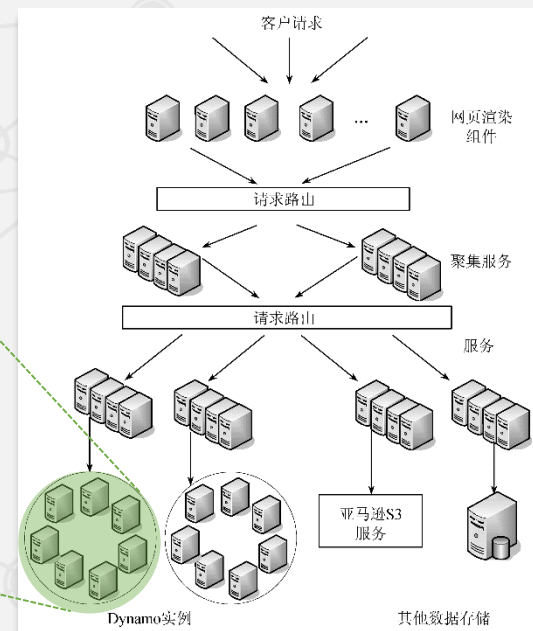
两个基本概念

preference list

存储与某个特定键值相对应的数据的节点列表

coordinator

执行一次读或写操作的节点



通常，coordinator 是 preference list 上的第一个节点

3.1 基础存储架构Dynamo

- Dynamo需要解决的主要问题及解决方案

问 题	采取的相关技术
1. 数据均衡分布	改进的一致性哈希算法
数据备份	参数可调的弱quorum机制
数据冲突处理	向量时钟 (Vector Clock)
成员资格及错误检测	基于Gossip协议的成员资格和错误检测
临时故障处理	Hinted handoff (数据回传机制)
永久故障处理	Merkle哈希树

3.1 基础存储架构Dynamo

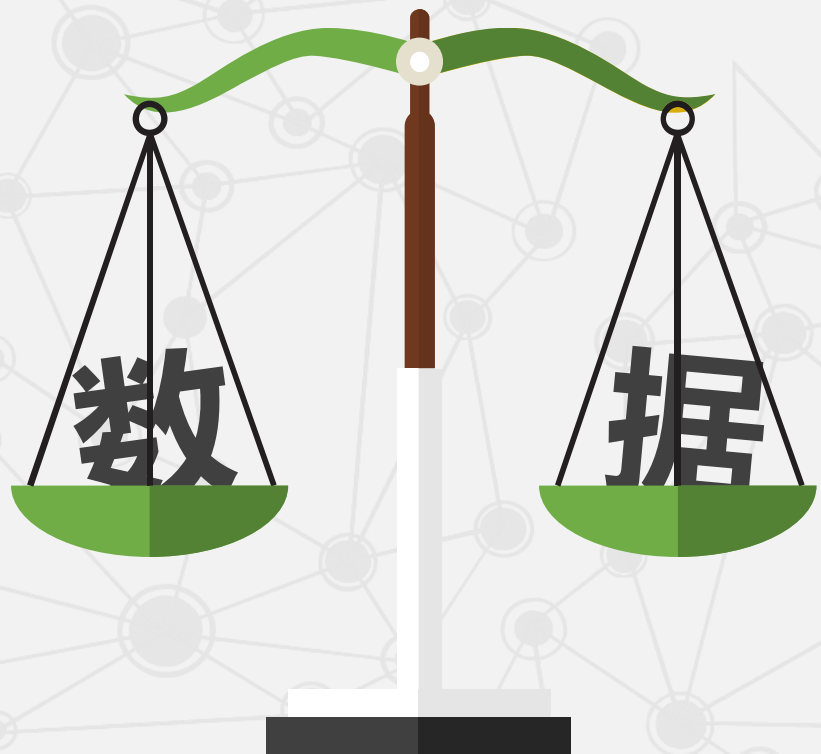
• 1. 数据均衡分布的问题

Dynamo采用了**分布式**的数据存储架构，均衡的数据分布可以保证负载平衡和系统良好的扩展性。

如何在各个节点上保证数据的均衡性，是影响Dynamo性能的关键问题。

Dynamo中使用**改进后的一致性哈希算法**，并在此基础上进行数据备份，以提高系统的可用性。

(GFS依靠Master进行负载均衡。)



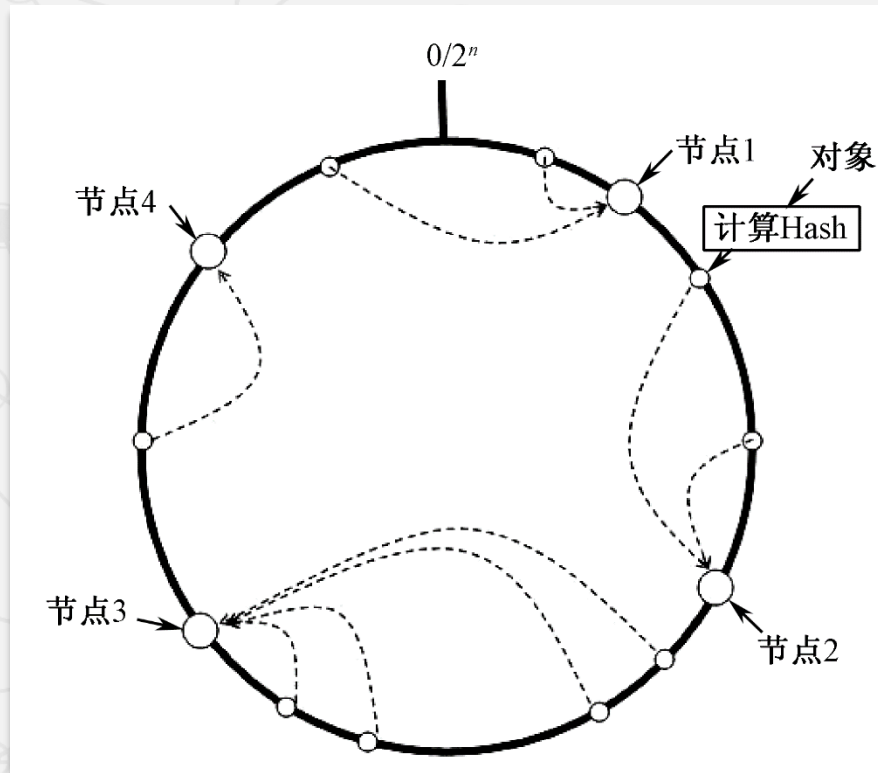
3.1 基础存储架构Dynamo

• 1. 数据均衡分布的问题

一致性哈希算法

一致性哈希算法是目前主流的**分布式哈希表** (Distributed Hash Table, DHT) 协议之一, 于**1997年**由**麻省理工学院**提出。

一致性哈希算法通过修正简单哈希算法, 解决了网络中的**热点问题**, 使得**DHT**可以真正地应用于**P2P环境**中。



3.1 基础存储架构Dynamo

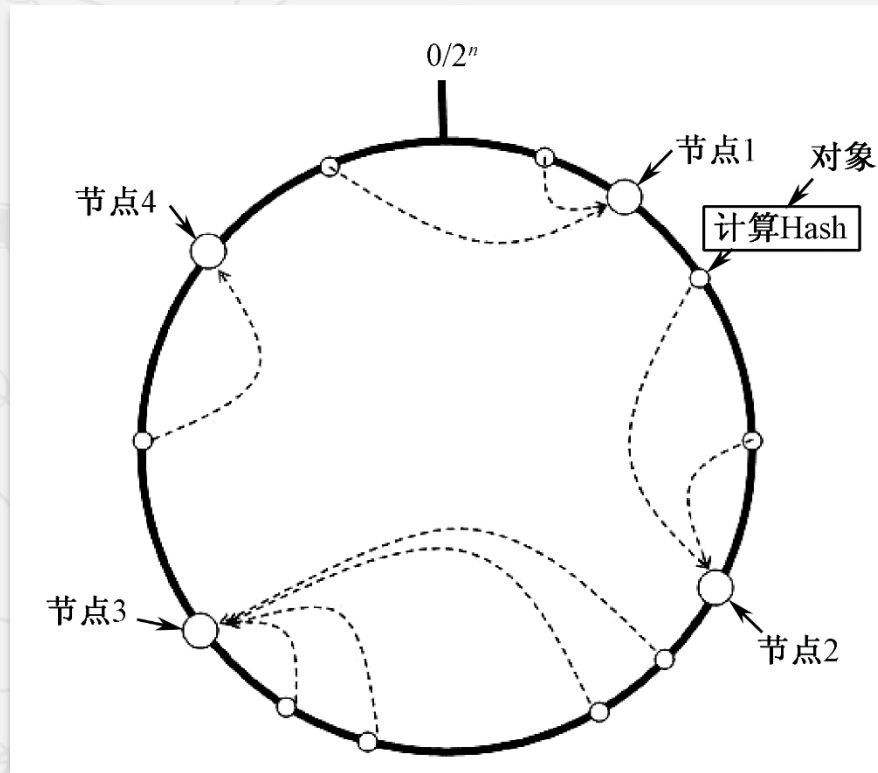
● 1. 数据均衡分布的问题

一致性哈希算法的基本过程为：

对于系统中的每个设备节点，为其分配一个随机标记，这些标记可以构成一个哈希环。

在存储数据时，计算出数据中键的哈希值，将其存放到哈希环顺时针方向上第一个标记大于或等于键的哈希值的设备上。

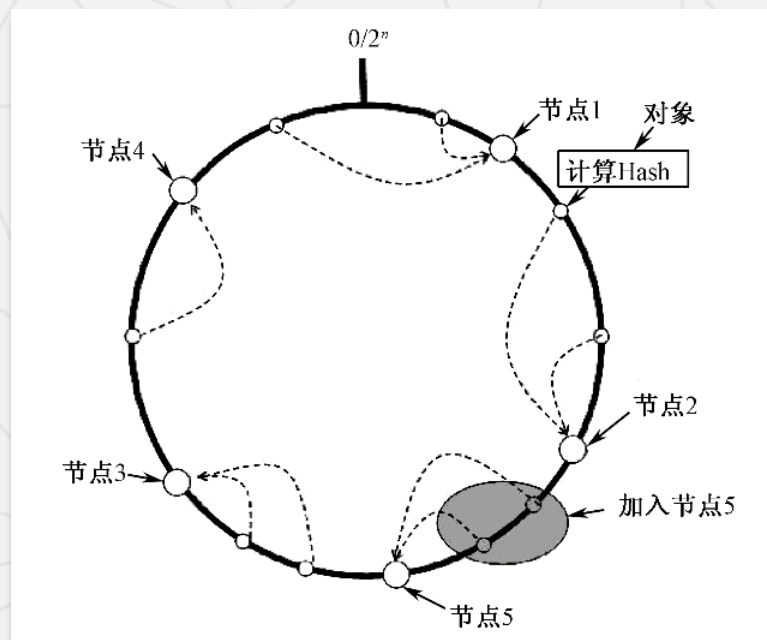
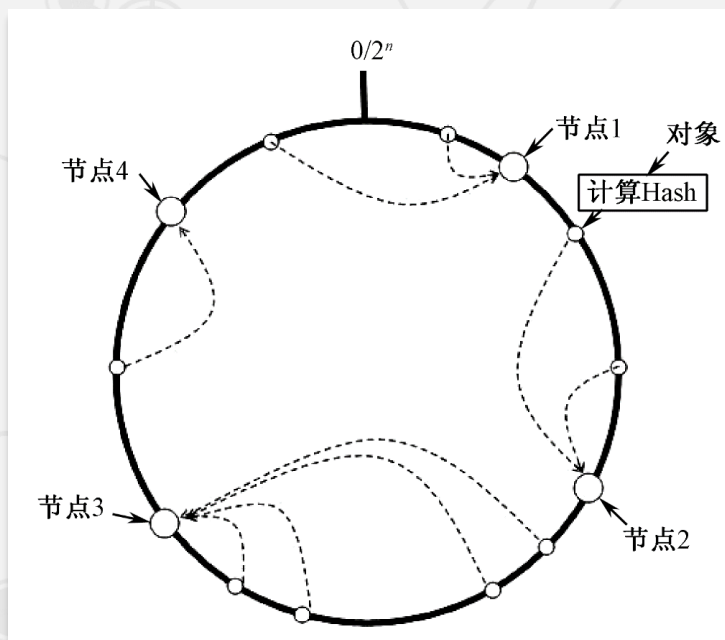
例如，某个数据的键的哈希值如右图的“对象”，按顺时针方向找到的第一个设备是节点2，则将该数据存放在节点2上。



3.1 基础存储架构Dynamo

1. 数据均衡分布的问题

一致性哈希算法除了能够保证哈希运算结果充分分散到整个环上外，还能保证在**添加或删除**设备节点时只会影响到其在哈希环中的**前驱设备节点**，而不会对其他设备节点产生影响。

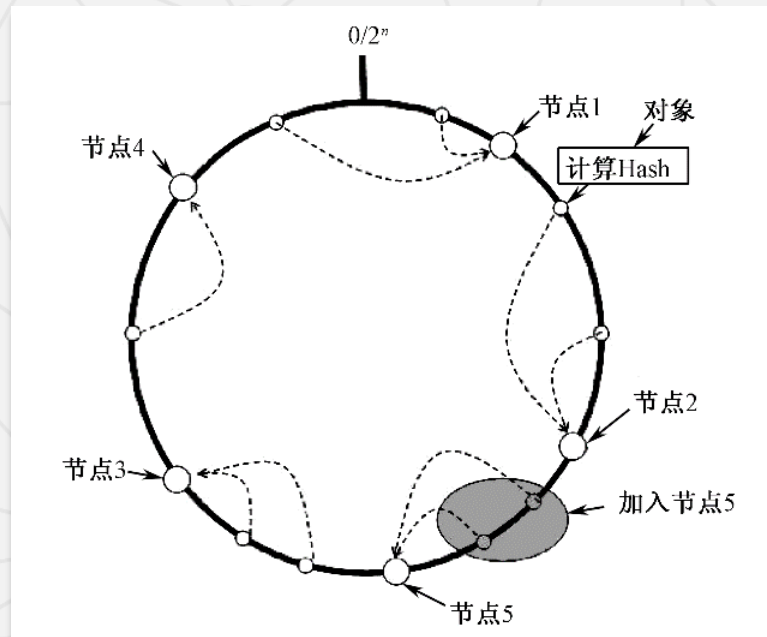
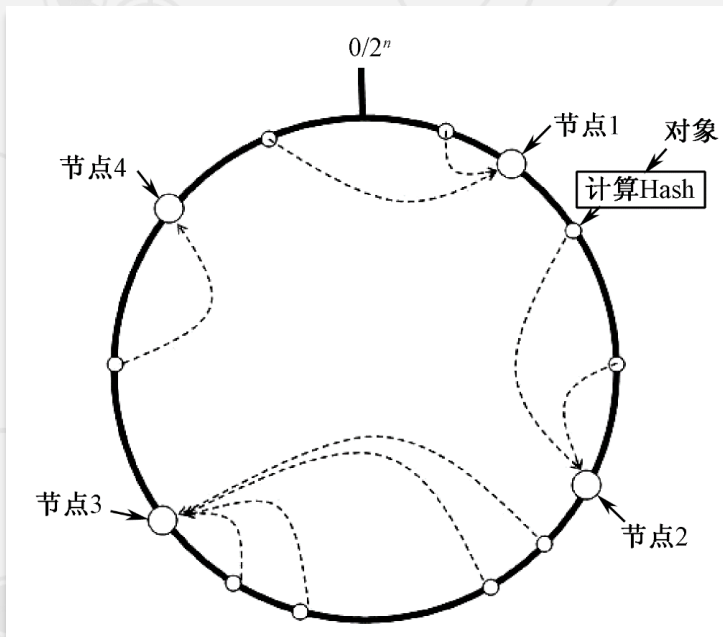


例如，在节点2和节点3之间增加节点5，则原来节点3上的部分数据会迁移到节点5，但节点1、2、4不会受到影响。

3.1 基础存储架构Dynamo

1. 数据均衡分布的问题

一致性哈希算法可以**大大降低**在添加或删除节点时引起的**节点间的数据传输开销**。



3.1 基础存储架构Dynamo

● 1. 数据均衡分布的问题

改进的一致性哈希算法

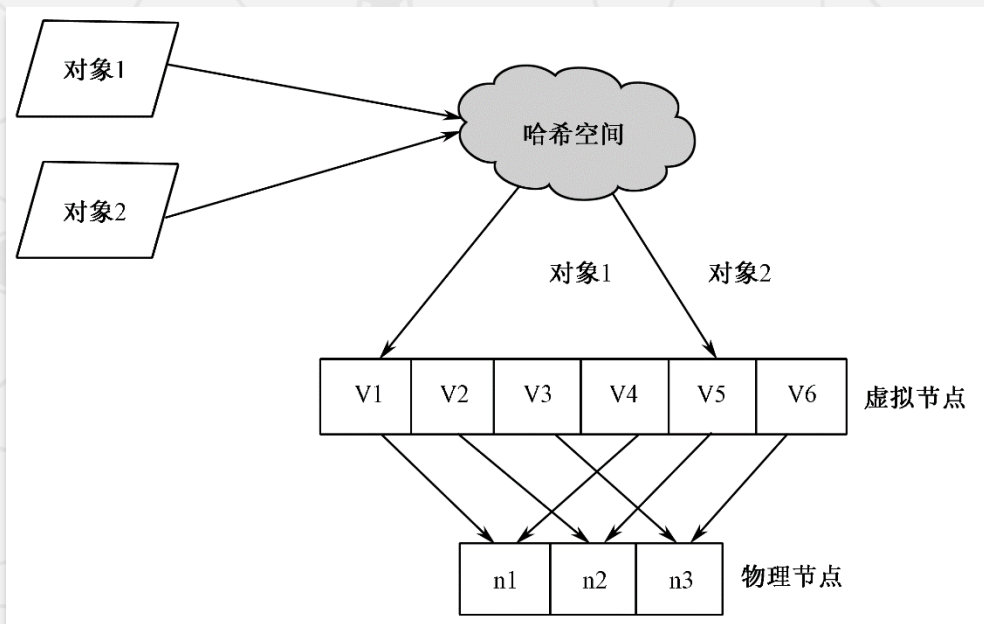
一致性哈希算法在设备节点数量较少的情况下，有可能造成环上节点分布的不均匀；并且没有考虑哈希环上不同设备节点的性能差异。

Dynamo中引入了**虚拟节点**的概念。

每个虚拟节点都**隶属于**某一个实际的**物理节点**，一个物理节点根据其性能的差异被分为**一个或多个虚拟节点**。

各个虚拟节点的**能力基本相当**，并**随机分布**在哈希环上。

存储时，数据对象先按照其键的哈希值，被分配到某个虚拟节点上，再存储在相应的物理节点上。



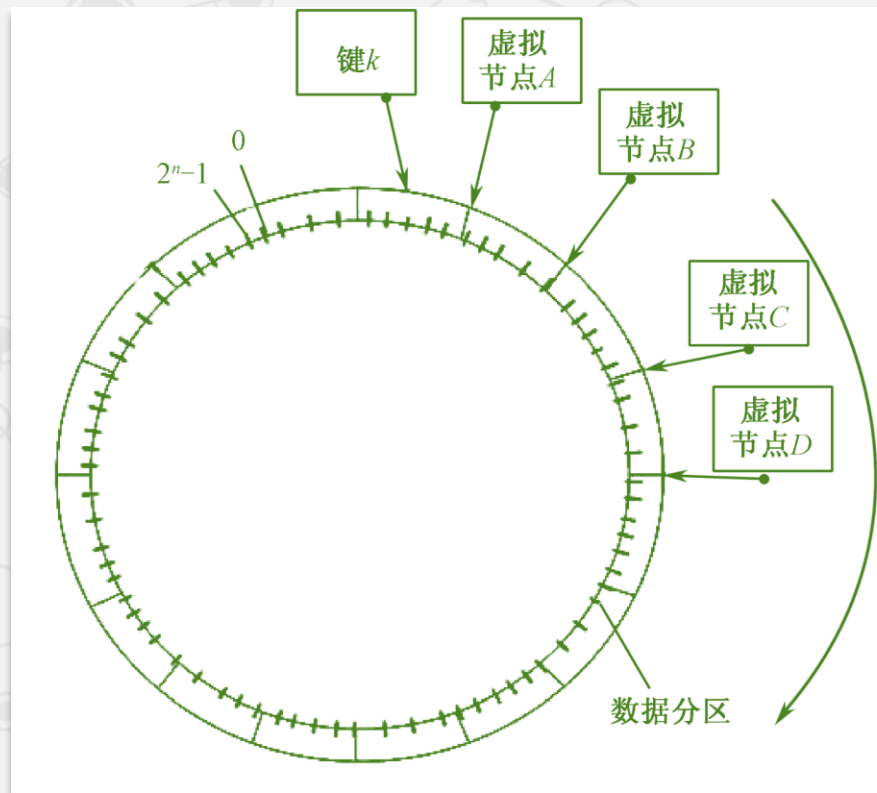
3.1 基础存储架构Dynamo

• 1. 数据均衡分布的问题

Dynamo将整个哈希环划分成 Q 等份，每个等份称为一个数据分区 (Partition)

假设系统中共有 S 个虚拟节点，且满足 $Q \gg S$ ，则每个虚拟节点负责的数据分区为 $V = Q/S$ 。

在存储数据时，每个数据先会被分配到某个数据分区，再根据负责该数据分区的虚拟节点，最终确定其所存储的物理节点。



3.1 基础存储架构Dynamo

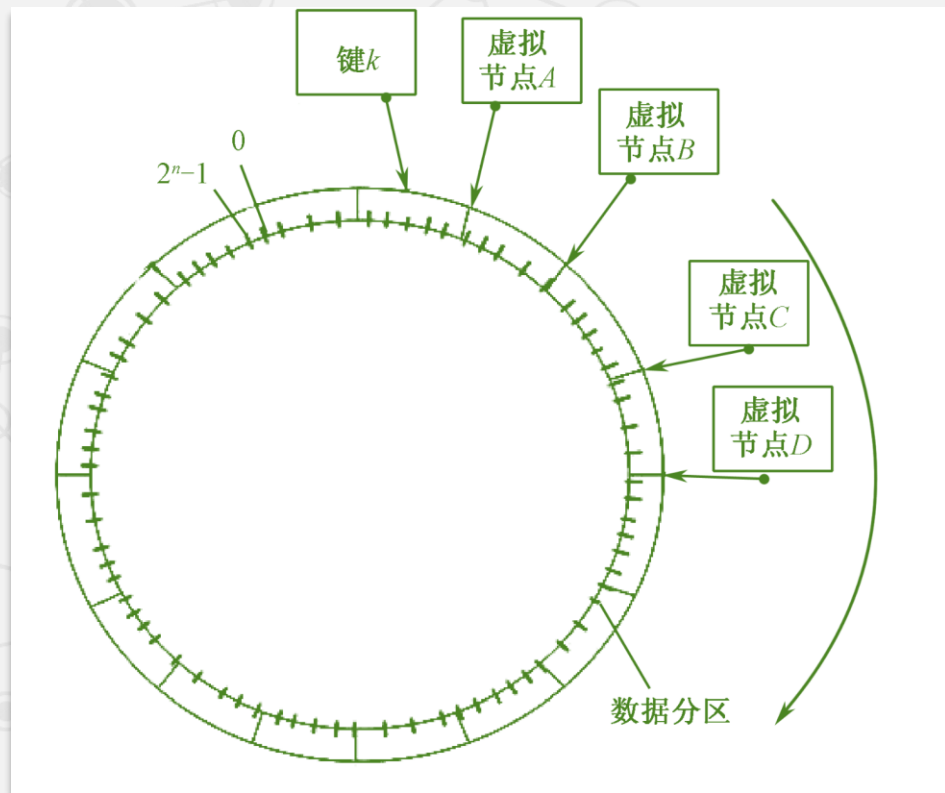
1. 数据均衡分布的问题

数据分区的好处有两点：

1 减小数据分布不均衡的可能性

在虚拟节点数量较少时，随机将数据分配到虚拟节点上，可能会引起数据的不均衡。由于数据分区远远大于虚拟节点的数量，可以减小数据分布不均匀的可能性。

2 添加或删除设备节点时引起较小的数据传输



3.1 基础存储架构Dynamo

1. 数据均衡分布的问题

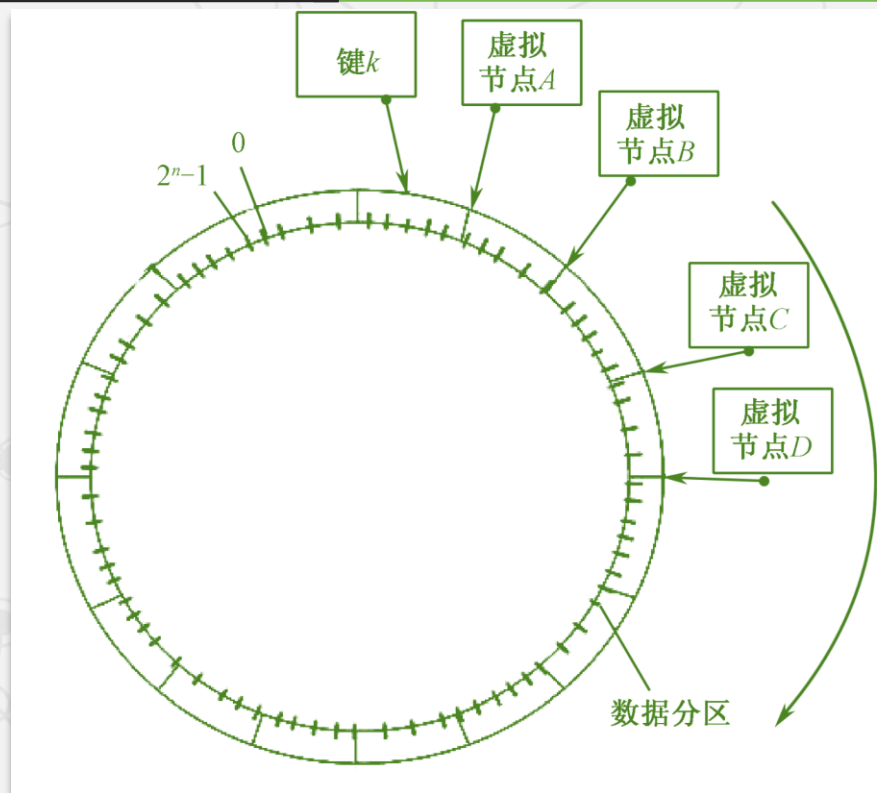
2 添加或删除设备节点时引起较小的数据传输

以添加设备为例，假设该设备节点包含N个虚拟节点。哈希环分成Q等份（数据分区），原有S个虚拟节点（ $Q \gg S$ ）。

原来，每个虚拟节点负责的数据分区数量为： Q/S

现在，每个虚拟节点负责的数据分区数量变为： $Q/(S + N)$

因此，有 $\frac{Q}{S} - \frac{Q}{S+N}$ 个数据分区的内容需要迁移到新的虚拟节点上



换个角度想，新增了N个虚拟节点，每个虚拟节点负责的数据分区数量为 $Q/(S + N)$ ，因此，有 $N \times \frac{Q}{S+N}$ 的数据需要传输。

3.1 基础存储架构Dynamo

1. 数据均衡分布的问题

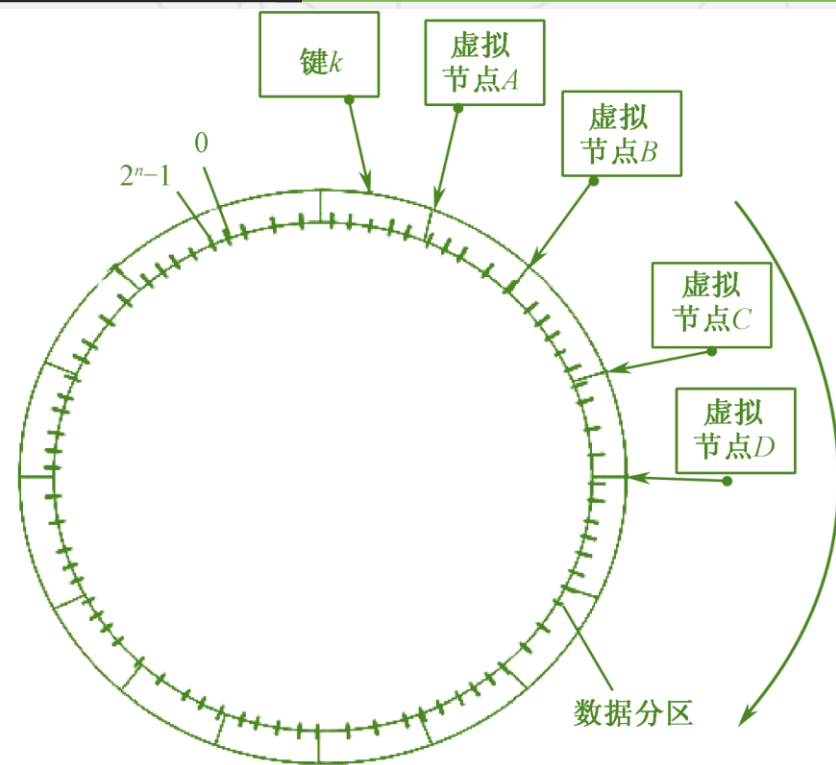
2

添加或删除设备节点时引起较小的数据传输

因此，在添加节点的过程中，只是改变了少量数据分区所属的虚拟节点，而并不需要改变大多数的数据分区。

这样，就可以在很小的数据传输代价下，保证整个系统中数据分布的均衡性。

需要注意的是，随着节点的增加，特别是 S （虚拟节点数）接近 Q （数据分区数）后，Dynamo的性能会急剧下降，因此，需要选择好 Q 的取值。



换个角度想，新增了 N 个虚拟节点，每个虚拟节点负责的数据分区数量为 $Q/(S + N)$ ，因此，有 $N \times \frac{Q}{S + N}$ 的数据需要传输。

3.1 基础存储架构Dynamo

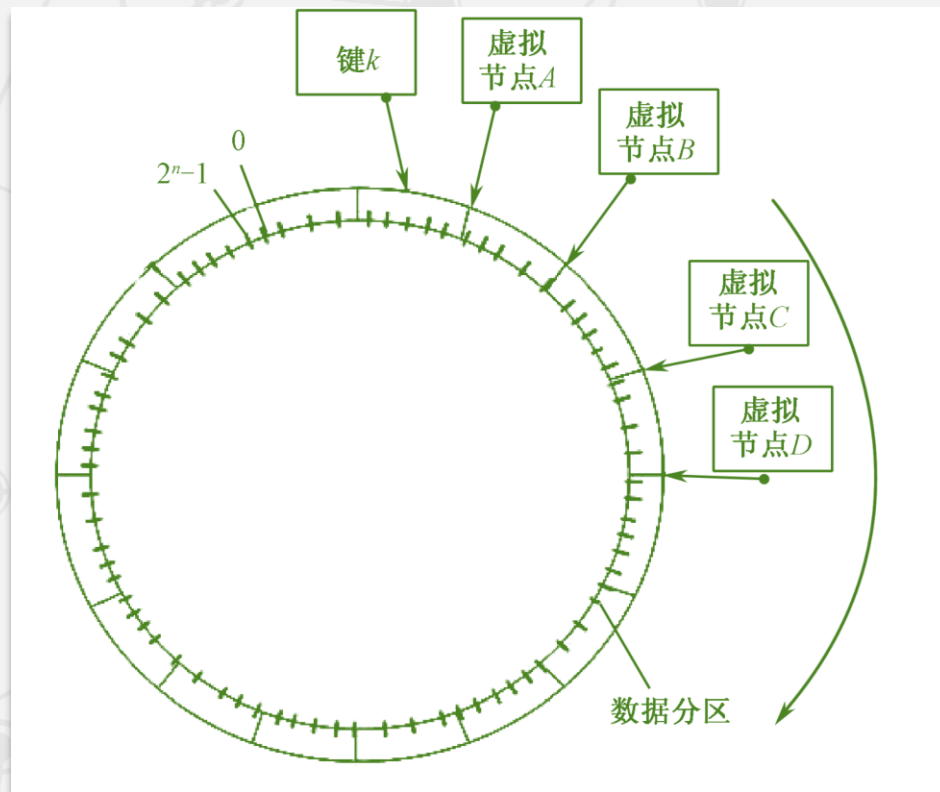
- **Dynamo**需要解决的主要问题及解决方案

问 题	采取的相关技术
数据均衡分布	改进的一致性哈希算法
2. 数据备份	参数可调的弱quorum机制
数据冲突处理	向量时钟 (Vector Clock)
成员资格及错误检测	基于Gossip协议的成员资格和错误检测
临时故障处理	Hinted handoff (数据回传机制)
永久故障处理	Merkle哈希树

3.1 基础存储架构Dynamo

● 2. 数据备份

为了提高数据的可用性，Dynamo中在存储每个数据对象时，保存了其多个副本作为冗余备份。



2.1 Google文件系统GFS

• Master容错

Master

命名空间 (Name Space) , 也就是整个文件系统的目录结构。

GFS通过操作日志来提供容错功能

Chunk与文件名的映射表。

Chunk副本的位置信息, 每一个Chunk默认有**三个副本**。

直接保存在各个Chunk Server上

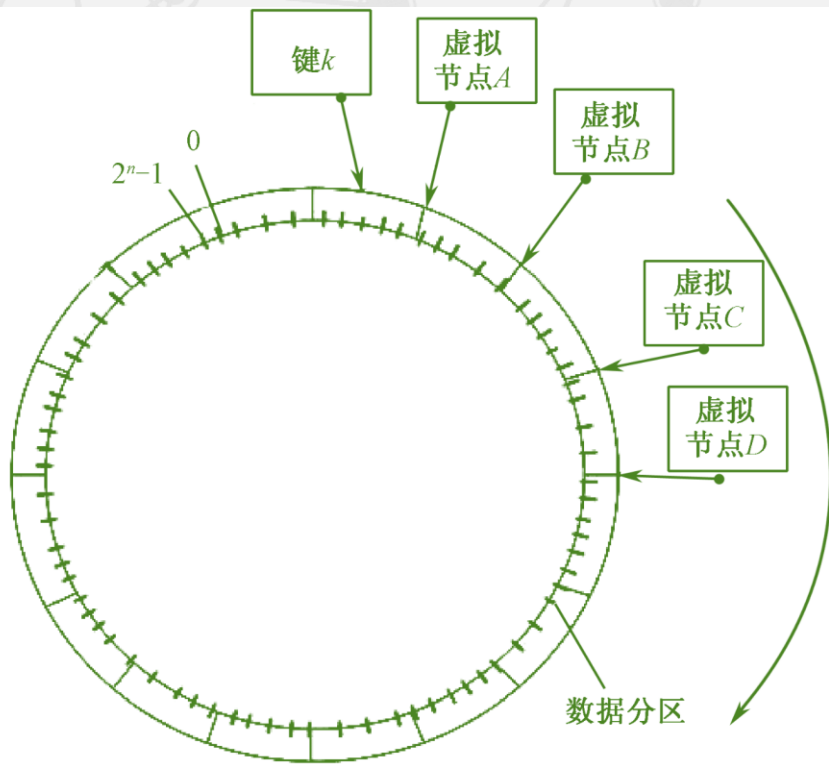
当Master发生故障时, 在磁盘数据保存完好的情况下, 可以迅速恢复以上元数据

为了防止Master彻底死机的情况, GFS还提供了Master**远程的实时备份**

3.1 基础存储架构Dynamo

● 2. 数据备份

为了提高数据的可用性，Dynamo中在存储每个数据对象时，保存了其多个副本作为冗余备份。



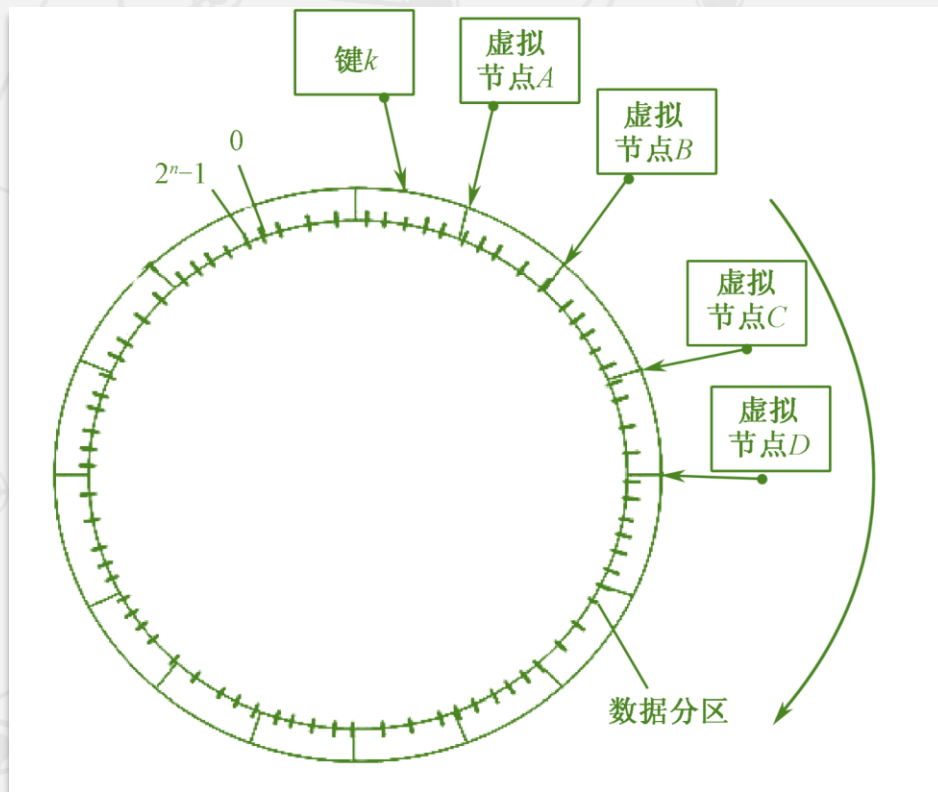
假设每个数据对象保存在系统中的副本数为 N （通常3），考虑到存在节点失效的情况，preference list中节点的个数大于 N ，并且为实际的物理节点。

在Dynamo中，每个数据的副本备份存储在哈希环顺时针方向上该数据所在虚拟节点的后继节点中。

3.1 基础存储架构Dynamo

● 2. 数据备份

如图所示，某个数据对象的键为 k ，其数据存储在虚拟节点A中，则其数据副本将按顺时针方向存储在节点B、C上。



数据备份在存储数据的**同时**进行，会使每次**写**操作的**延时**变长。

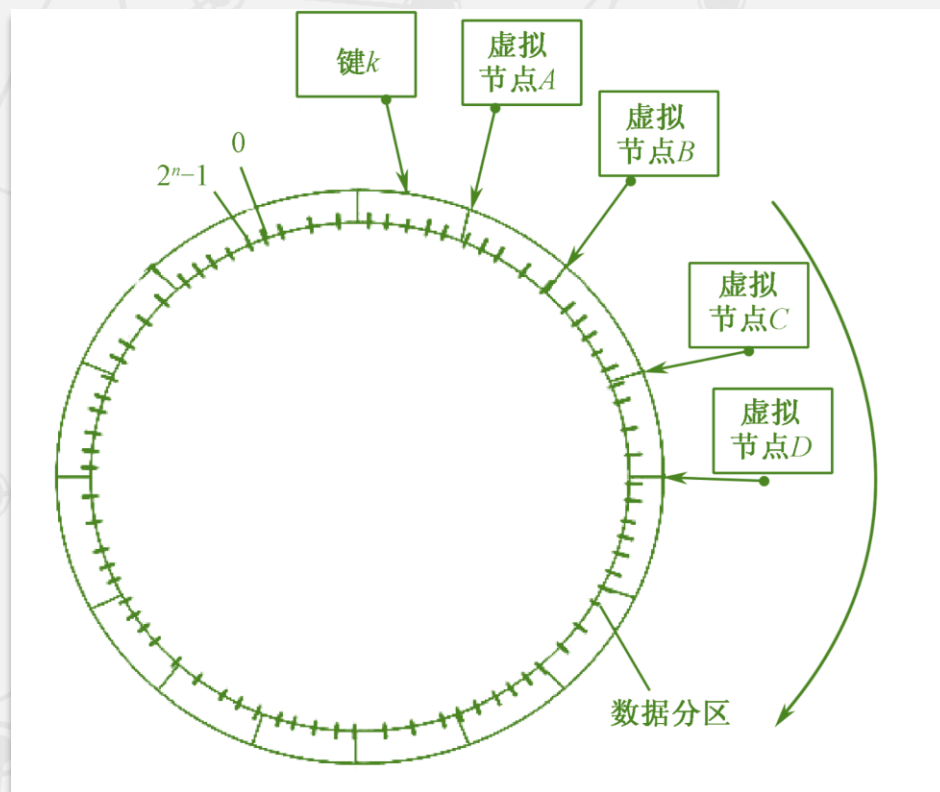
因此，Dynamo对写操作进行了优化，保证**一个**副本必须写入**硬盘**，其他副本只要写入节点的**内存**即返回“**写成功**”。这样，既保证了副本数量，又减少了时延。

根据这个规定，每个虚拟节点上实际存储了分配给它，以及它的前 **$N-1$** 个**前驱虚拟节点**的数据。

3.1 基础存储架构Dynamo

● 2. 数据备份

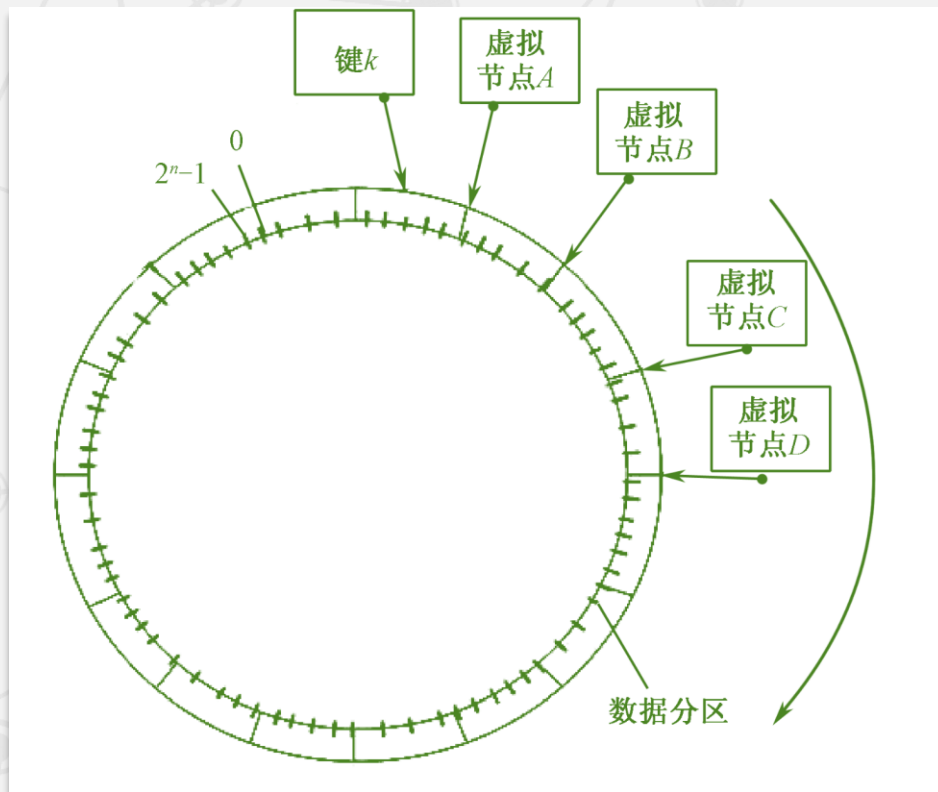
此外，Amazon还保证相邻的虚拟节点分别位于**不同的地理区域**，这样，即便某个数据中心由于自然灾害或断电的原因整体瘫痪，仍然可以保证其他数据中心的备份。



3.1 基础存储架构Dynamo

● 2. 数据备份

Dynamo在产生N个数据副本时，采用了参数可调的**弱quorum (Sloppy quorum)**的机制（课本25页提到过quorum机制）。该机制中涉及三个参数W、R、N。



其中，W表示一次成功的写操作至少需要写入的副本。

R表示一次成功读操作须由服务器返回给用户的最小副本数。

N表示每个数据存储的副本数。

只要满足 $R + W > N$ ，就可以保证在故障的节点数量不超过1台时，用户至少可以获得一份最新的数据副本。

3.1 基础存储架构Dynamo

● 2. 数据备份

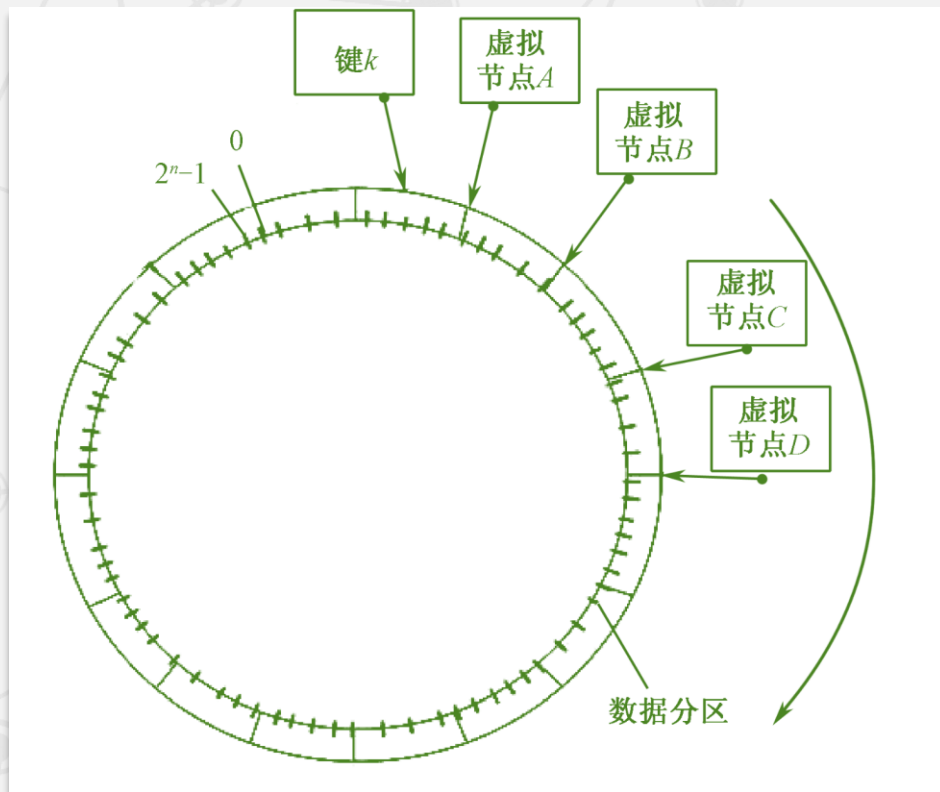
通过配置R和W，可以调节系统的性能。

如果应用要求较高的**读效率**，则可以设置 $R=1$ ， $W=N$ ；

如果应用要求较高的**写效率**，则可以设置 $R=N$ ， $W=1$ ；

如果希望在**读/写效率**中取得**平衡**，以 $N=3$ 为例，则可以设置 $R=2$ ， $W=2$ 。

此外，通过配置R和W，还可以在系统的**可用性和容错性**之间取得**平衡**。例如，当 $N=3$ 时，如果丢失一些最后的更新，并不会造成太大的影响，则可以 $R=1$ ， $W=1$ 。



3.1 基础存储架构Dynamo

- Dynamo需要解决的主要问题及解决方案

问 题	采取的相关技术
数据均衡分布	改进的一致性哈希算法
数据备份	参数可调的弱quorum机制
3. 数据冲突处理	向量时钟 (Vector Clock)
成员资格及错误检测	基于Gossip协议的成员资格和错误检测
临时故障处理	Hinted handoff (数据回传机制)
永久故障处理	Merkle哈希树

3.1 基础存储架构Dynamo

• 3. 数据冲突问题

分布式系统架构中通常考虑的三个因素

可靠性
(Reliability)

可用性
(Availability)

一致性
(Consistency)

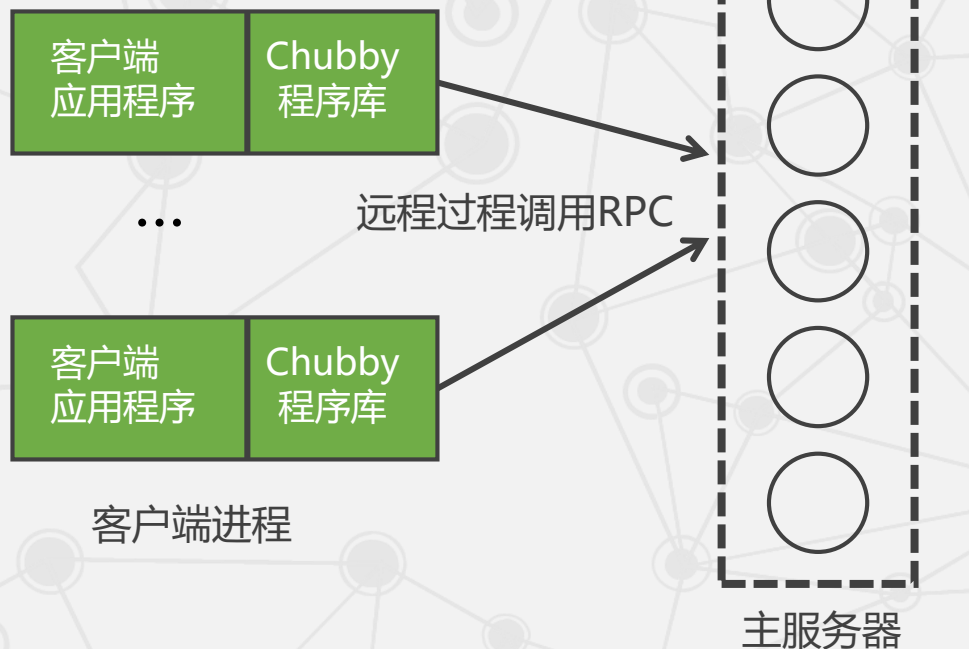
分布式系统架构中通常需要考虑三个因素：可靠性（Reliability）、可用性（Availability）和一致性（Consistency）。但这三者**不能同时满足**，最多只能实现其中的两个。

Dynamo系统根据其业务特点，选择通过**牺牲一致性**来保证系统的**可靠性和可用性**。

2.3 分布式锁服务Chubby

• Chubby的基本架构

Google靠Chubby来保障一致性。



客户端

在客户这一端每个客户应用程序都有一个Chubby程序库 (Chubby Library)，客户端的所有应用都是通过调用这个库中的相关函数来完成的。

服务器端

服务器端称为Chubby单元，一般是由五个副本 (Replica) 服务器组成，这五个副本在配置上完全一致，并且在系统刚开始时处于对等地位。

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

三个
条件

决议只有在被proposers提出后才能批准

每次只批准一个决议

只有决议确定被批准后learners才能获取这个决议

p1: 每个acceptor只接受它得到的第一个决议。

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

p2b: 一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

p2c: 如果一个编号为n的提案具有值v，那么存在一个“多数派”，要么它们中没有谁批准过编号小于n的任何提案，要么它们进行的最近一次批准具有值v。

当且仅当 acceptors 没有收到编号大于n的请求时，acceptors 才批准编号为n的提案。

3.1 基础存储架构Dynamo

• 3. 数据冲突问题

分布式系统架构中通常考虑的三个因素

可靠性
(Reliability)

可用性
(Availability)

一致性
(Consistency)

分布式系统架构中通常需要考虑三个因素：可靠性（Reliability）、可用性（Availability）和一致性（Consistency）。但这三者不能同时满足，最多只能实现其中的两个。

Dynamo系统根据其业务特点，选择通过**牺牲一致性**来保证系统的**可靠性和可用性**。

Dynamo没有采用**强一致性模型**（Strong Consistency），而采用了**最终一致性模型**（Eventual Consistency）。

3.1 基础存储架构Dynamo

• 3. 数据冲突问题

分布式系统架构中通常考虑的三个因素

可靠性
(Reliability)

可用性
(Availability)

一致性
(Consistency)

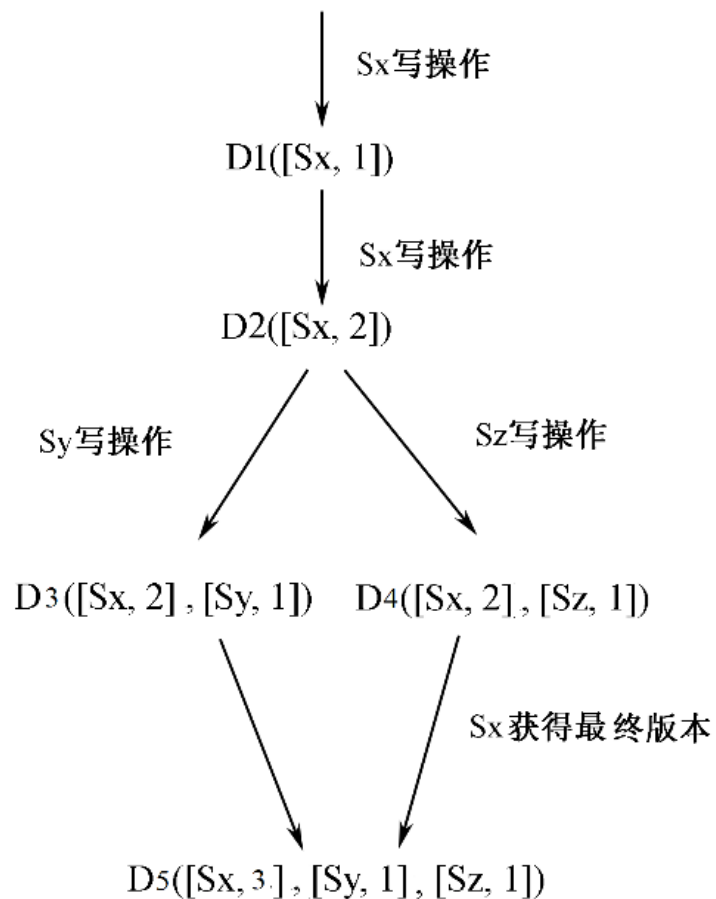
最终一致性模型 (Eventual Consistency) 不要求各个数据副本在更新过程中始终保持一致，只需要**最终时刻**，所有数据副本能够保证一致性。

由于Dynamo中可能出现同一个数据被**多个节点同时更新**的情况，且无法保证数据副本的**更新顺序**，这有可能会**导致数据冲突**。

为了解决该问题，Dynamo中采用了**向量时钟技术** (Vector Clock) 。

3.1 基础存储架构Dynamo

• 3. 数据冲突问题



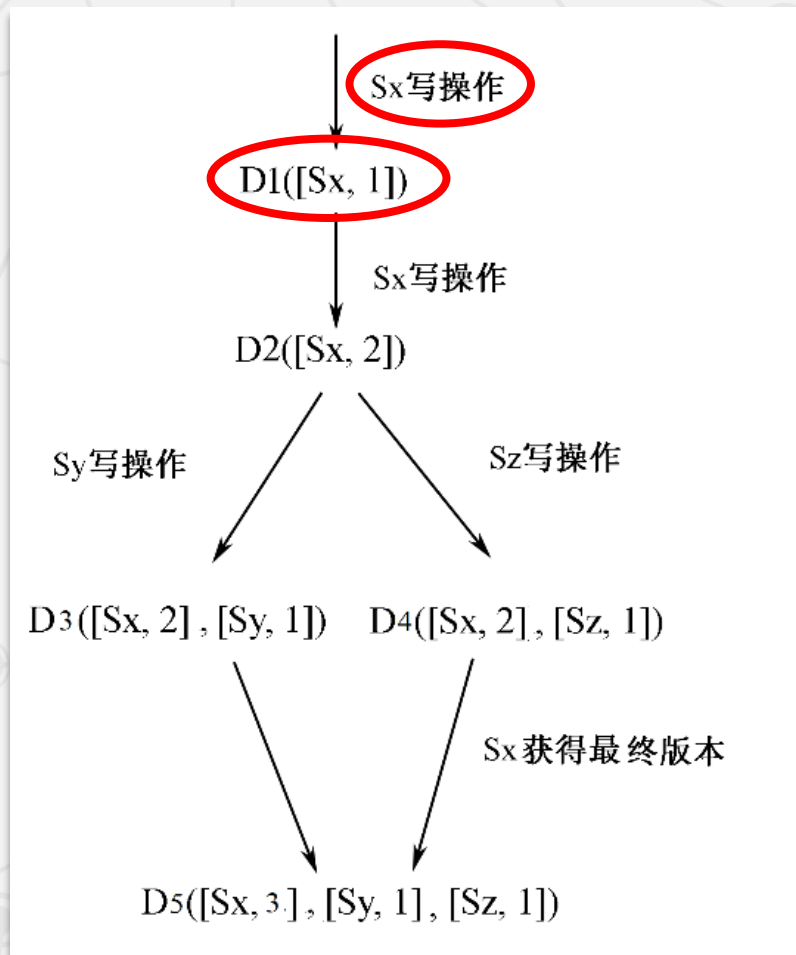
图中展示了一个向量时钟的示例。

Dynamo中的向量时钟通过`[node, counter]`对来表示。

其中，**node**表示**操作节点**，**counter**是其对应的**计数器**，**初始值为 0**，节点每进行一次更新操作则**计数器加 1**。

3.1 基础存储架构Dynamo

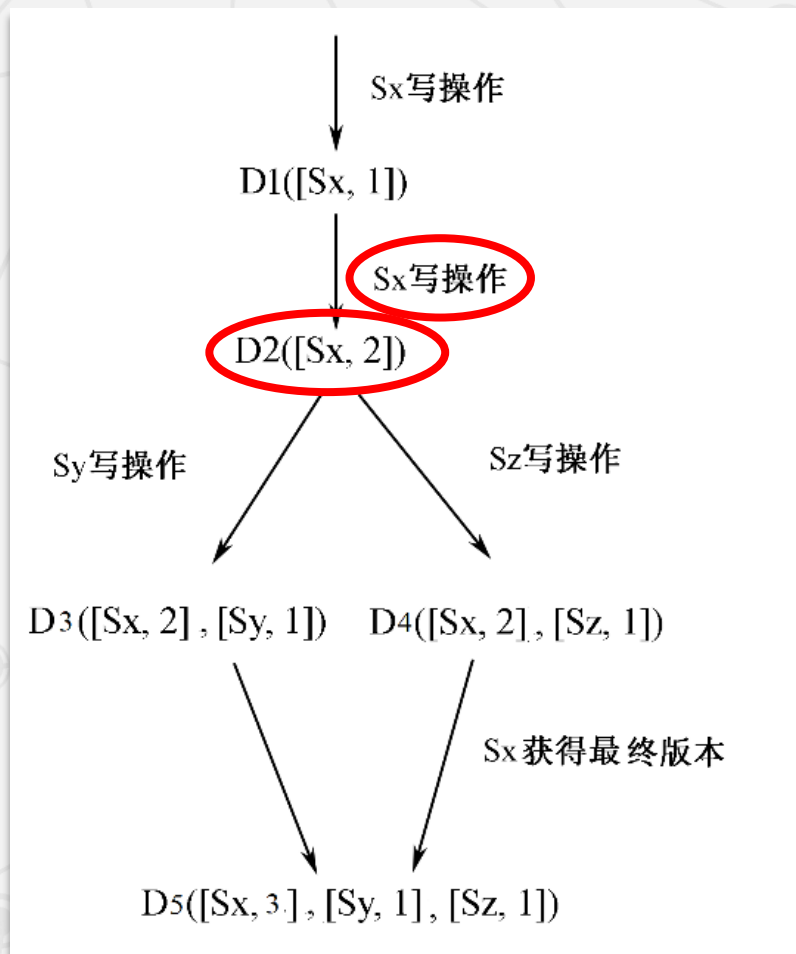
● 3. 数据冲突问题



- 如图所示，节点 S_x 首先对数据对象 D 进行了一次更新操作，产生了第一个版本 $D1[S_x, 1]$ 。其中，操作节点为 S_x ，计数器的值为1。

3.1 基础存储架构Dynamo

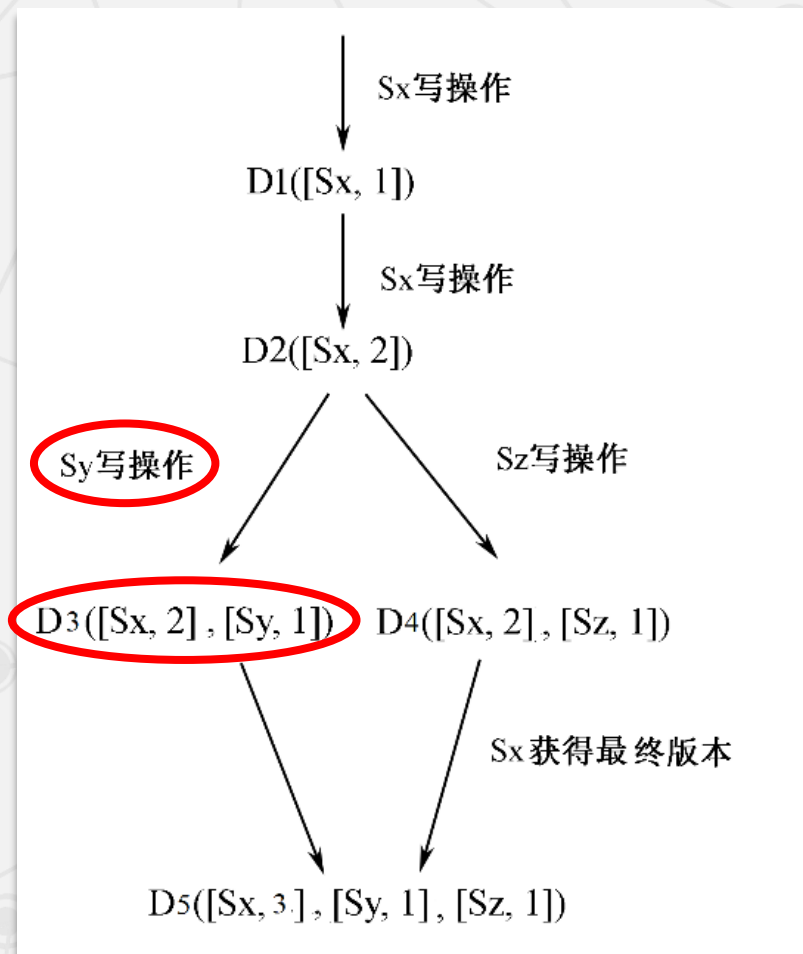
● 3. 数据冲突问题



- 如图所示，节点Sx首先对数据对象D进行了一次更新操作，产生了第一个版本D1[Sx, 1]。其中，操作节点为Sx，计数器的值为1。
- 接着，Sx进行了第二次操作，产生第二个版本D2[Sx, 2]。其中，操作节点不变，计数器加1。

3.1 基础存储架构Dynamo

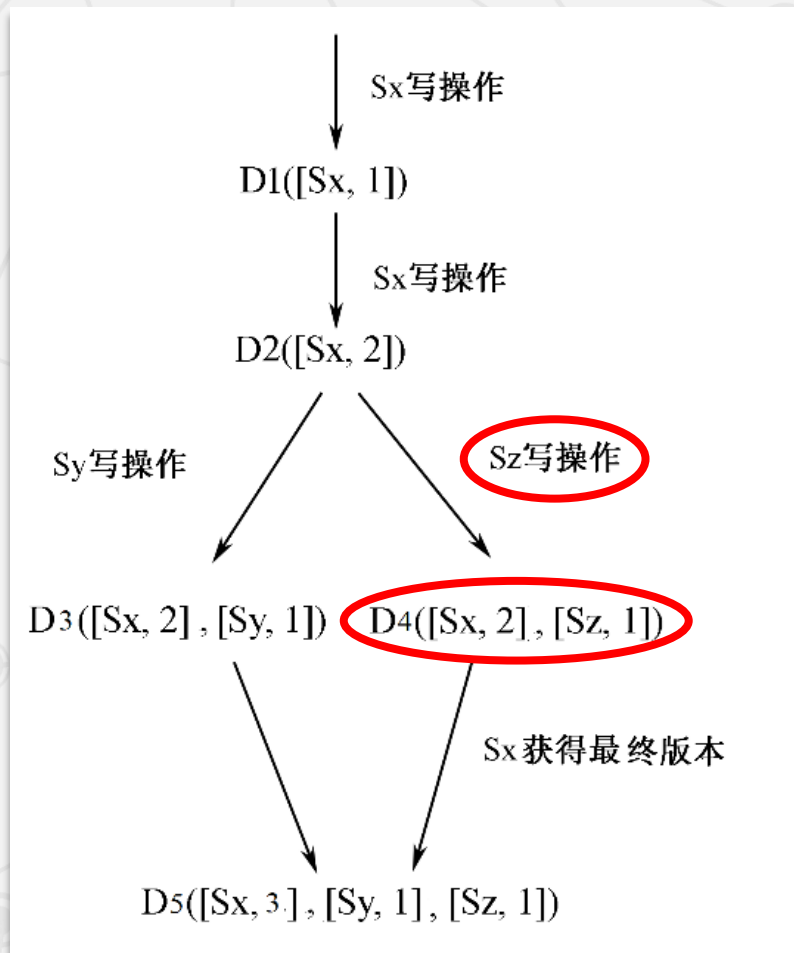
3. 数据冲突问题



- 如图所示，节点Sx首先对数据对象D进行了一次更新操作，产生了第一个版本D1[Sx, 1]。其中，操作节点为Sx，计数器的值为1。
- 接着，Sx进行了第二次操作，产生第二个版本D2[Sx, 2]。其中，操作节点不变，计数器加1。
- 之后，节点Sy和Sz分别同时对该数据对象进行更新操作，Sy将自身的信息加入时钟向量，产生了新的版本D3([Sx, 2], [Sy, 1]);

3.1 基础存储架构Dynamo

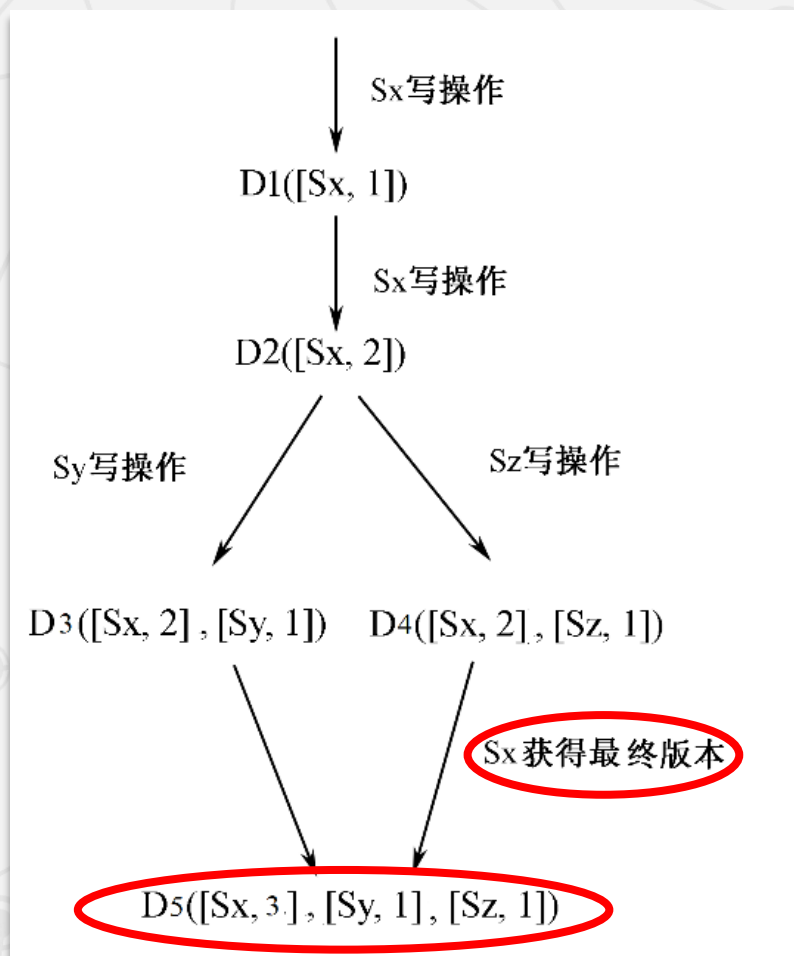
● 3. 数据冲突问题



- 同样，Sz也产生了新的版本D4([Sx, 2], [Sz, 1])。
- 此时，对于同一个数据对象D。系统中存在两个相互冲突的版本。

3.1 基础存储架构Dynamo

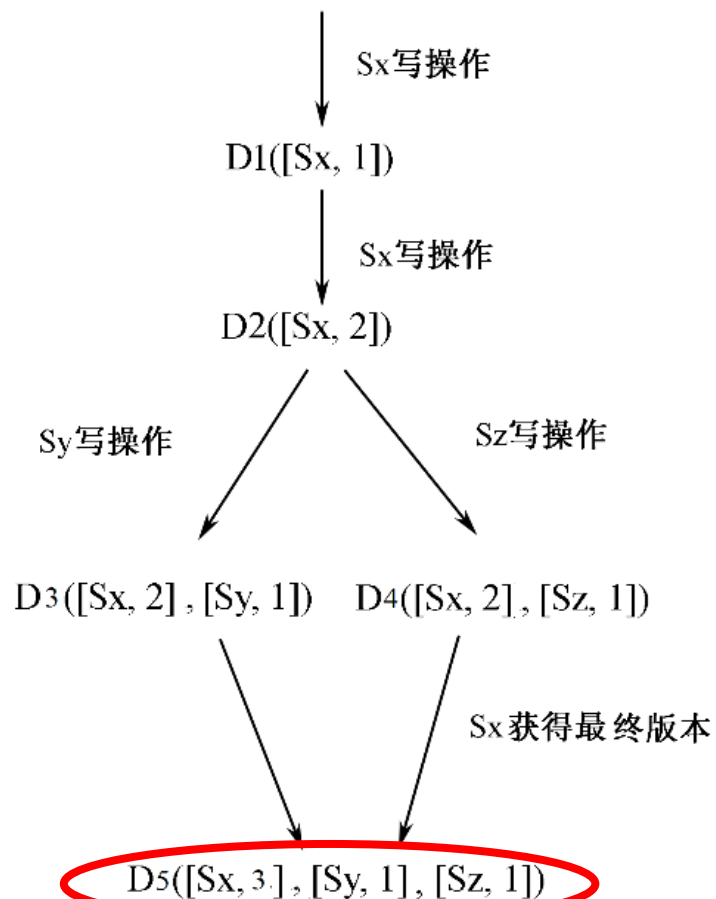
• 3. 数据冲突问题



- 同样，Sz也产生了新的版本D4([Sx, 2], [Sz, 1])。
- 此时，对于同一个数据对象D。系统中存在两个相互冲突的版本。
- 如果节点Sx需要再次更新数据对象D，它将会同时获得两个数据版本，并通过解决数据冲突来获得最终版本，如D5([Sx, 2], [Sy, 1], [Sz, 1])。

3.1 基础存储架构Dynamo

• 3. 数据冲突问题



- 常用的解决冲突的方案有两种：（1）通过客户端由**用户**解决；（2）系统自动**选择时间戳**最近的版本。
- 但由于集群内的各个节点并不能严格保证时钟同步，所以，不能完全保证最终版本的准确性。
- 需要注意的是，向量时钟的数量是有限制的，当超过限制时，会根据时间戳删除最早的向量时钟。

3.1 基础存储架构Dynamo

- Dynamo需要解决的主要问题及解决方案

问 题	采取的相关技术
数据均衡分布	改进的一致性哈希算法
数据备份	参数可调的弱quorum机制
数据冲突处理	向量时钟 (Vector Clock)
4. 成员资格及错误检测	基于Gossip协议的成员资格和错误检测
临时故障处理	Hinted handoff (数据回传机制)
永久故障处理	Merkle哈希树

3.1 基础存储架构Dynamo

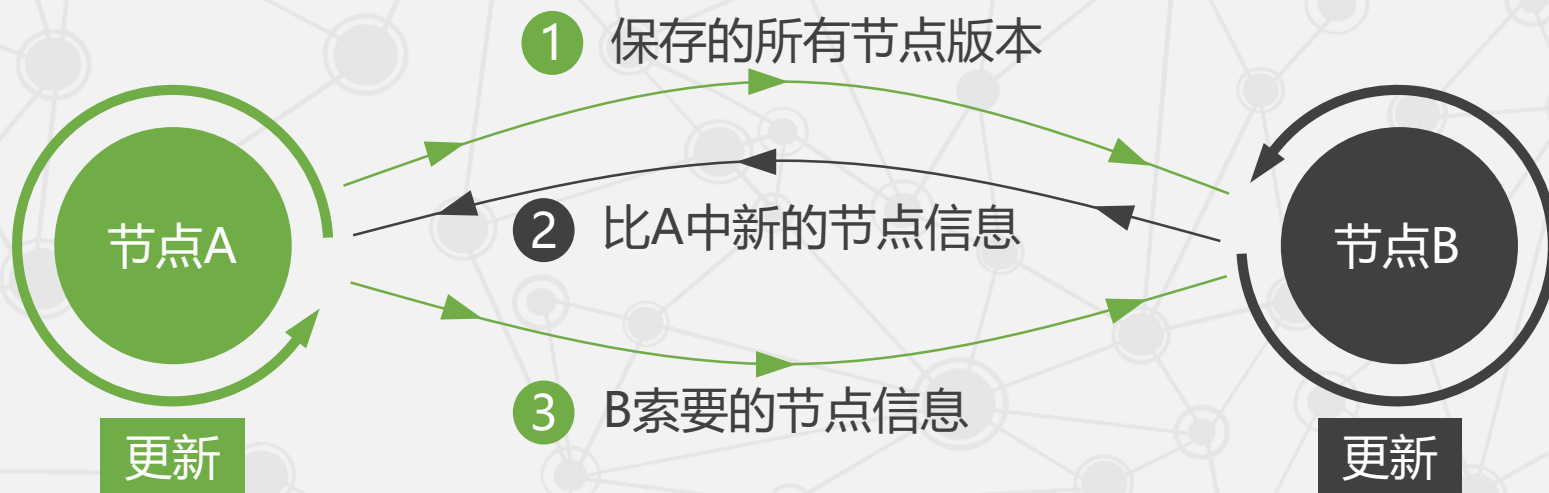
● 4. 成员资格及错误检测

- 由于Dynamo采用了**无中心**的架构，每个成员节点都需要保存其他节点的**路由信息**，以保障系统中各个**节点之间数据转发**顺畅。
- 但由于机器或人为因素，Dynamo中**添加或删除**节点的情况时常发生。为了保证每个节点都能拥有最新的成员节点信息，Dynamo中采用了一种类似于**Gossip（闲聊）协议**的技术。
- **Gossip（闲聊）协议**要求每个节点相隔固定时间（1秒），就从其他节点中任意选择一个与之通信。
- 如果通信时，连接成功，则双方交换各自保存的系统中节点的负载、路由等信息。

3.1 基础存储架构Dynamo

● 4. 成员资格及错误检测

1. 在交换信息时，一个节点A先将保存的所有节点版本发送给对方节点B；
2. 节点B将收到的节点版本与自身保存节点版本相比对，将比A中新的节点信息发送给A，同时，告知A需要将哪些节点信息发给它；
3. 节点A收到节点B的回复后，更新自身的节点信息，并将B索要的节点信息发给B；
4. 节点B接受A发来的节点信息，并更新自身的节点信息。

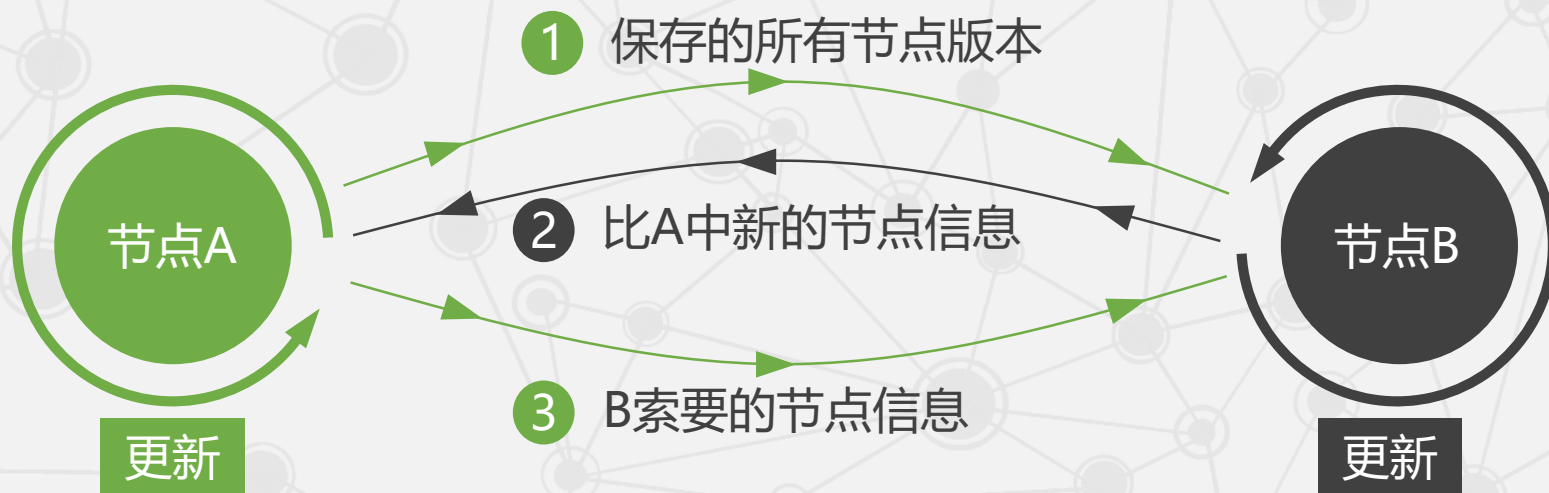


这样节点A和节点B就完成了节点信息的互换，同时更新各自保存的节点信息。

3.1 基础存储架构Dynamo

● 4. 成员资格及错误检测

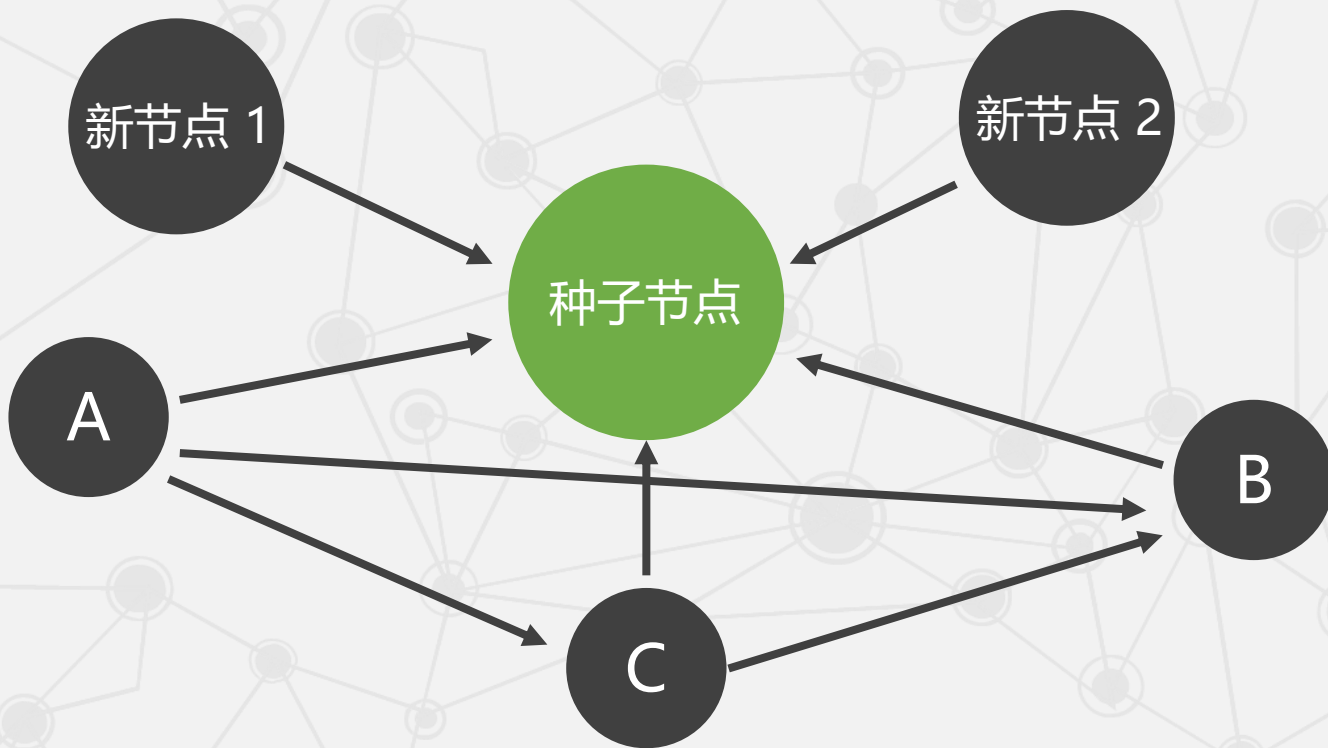
- Dynamo中还通过Gossip来实现错误检测。
- 任何节点向其他节点发起通信后，如果对方没有回应，则认为对方节点失效，并选择别的节点进行通信。
- 发起通信的节点会定期向失效节点发出消息，如果对方回应，则可以重新建立通信。



3.1 基础存储架构Dynamo

● 4. 成员资格及错误检测

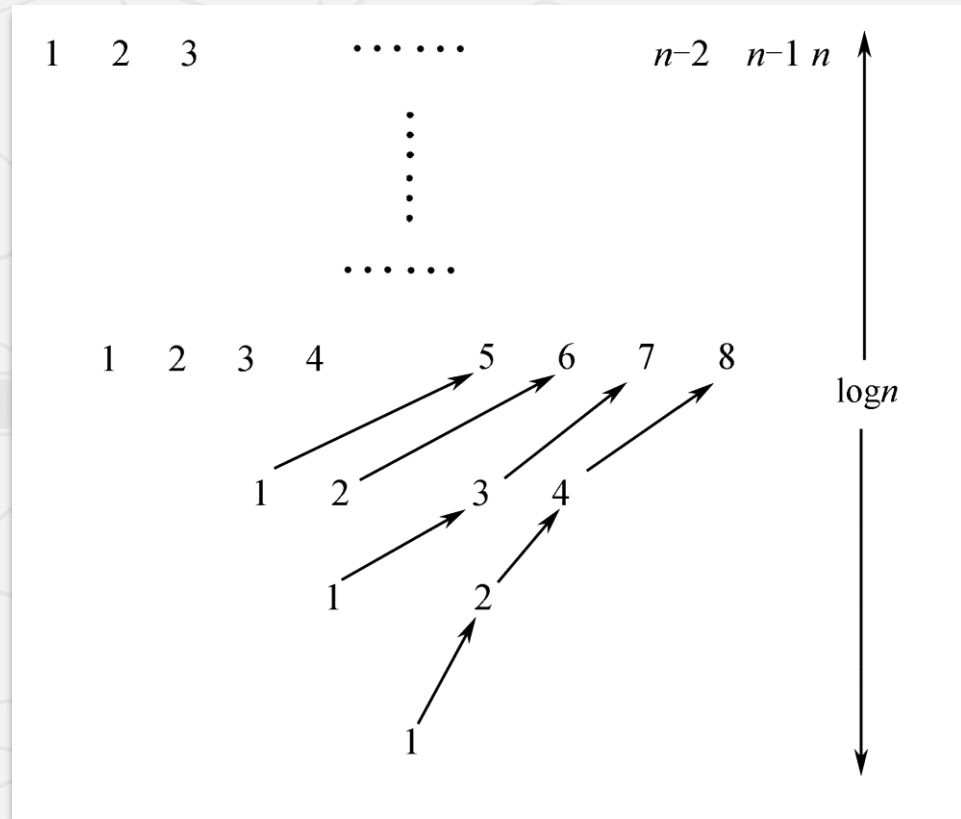
- 为了避免新加入的节点之间不能及时发现其他节点的存在，Dynamo中设置了一些**种子节点 (Seed Node)**。
- 种子节点和**所有**的节点都有联系。当新节点加入时，它扮演一个**中介**的角色，使新加入节点之间互相感知。



3.1 基础存储架构Dynamo

● 4. 成员资格及错误检测

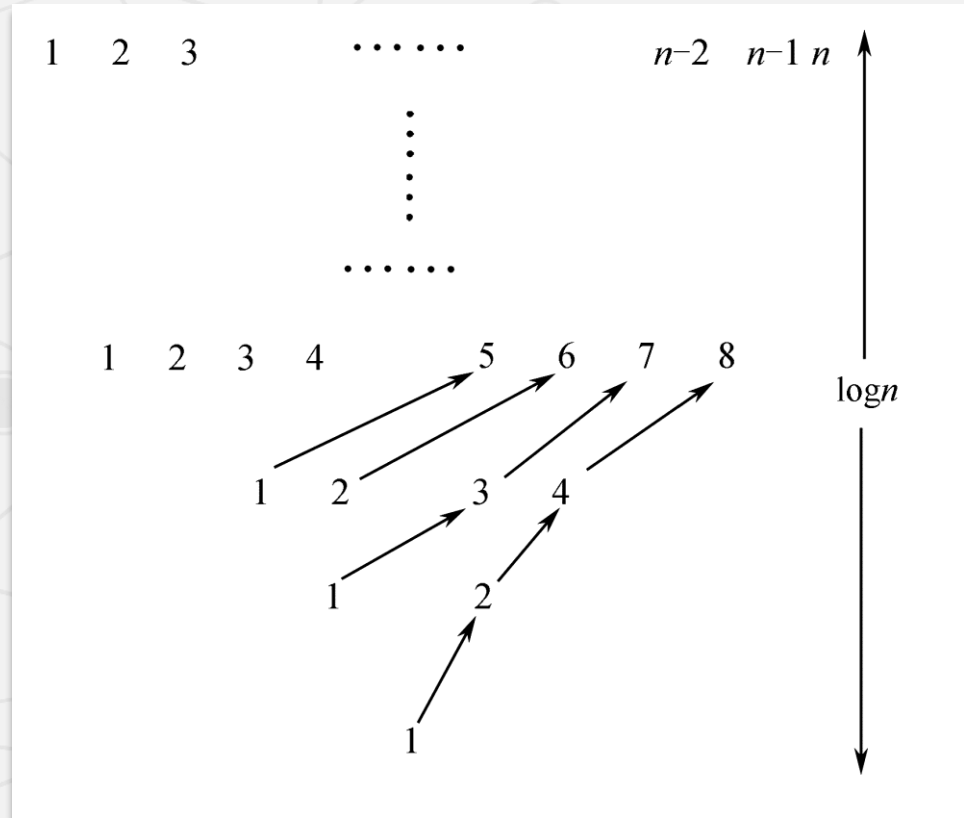
- 假如一新节点加入节点总数为 n 的系统，并以最优的方式进行传播，即每次通信的两个节点都是第一次交换新节点信息。
- 那么，将新节点信息传遍整个系统需要的时间复杂度为 $\log_2 n$ （可以理解为通信的“层数”）。



3.1 基础存储架构Dynamo

● 4. 成员资格及错误检测

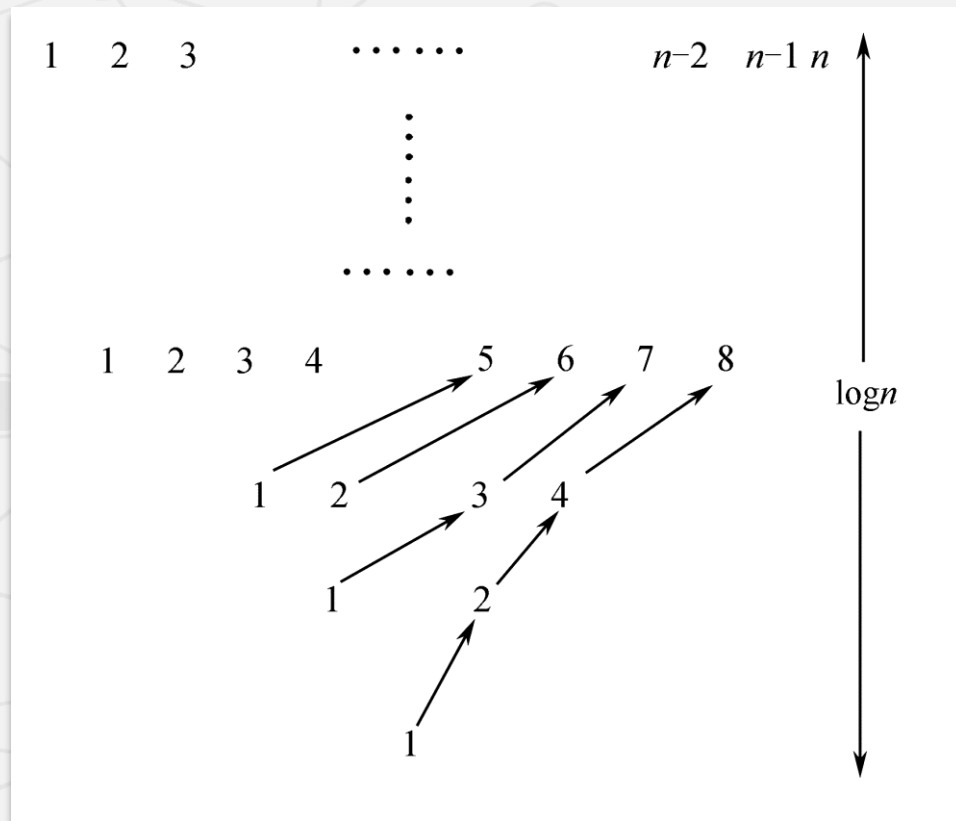
- 当 $n = 2$ 时，即只有2个节点时，只需要一次通信，就能交换完信息。（ $2^x = 2, x = 1$ ）
- 当 $n = 4$ 时，有4个节点时，需要2层通信，才能交换完信息。（ $2^x = 4, x = 2$ ）
- 以此类推，当有 n 个节点时，需要 x 层通信，才能交换完信息，（ $2^x = n, x = \log_2 n$ ），即时间复杂度为 $\log_2 n$ 。



3.1 基础存储架构Dynamo

● 4. 成员资格及错误检测

- 显然，当 n 的值很大时，传播时间就会变得很长。
- 因此，Dynamo中的节点数不能太多。
- 根据Amazon的实际经验，当节点数 n 在数千时，Dynamo的效率非常高；
- 但当节点数 n 增加到数万后，效率就会急剧下降。
- 为此，Amazon采用了分层Dynamo结构来解决该问题



3.1 基础存储架构Dynamo

- Dynamo需要解决的主要问题及解决方案

问 题	采取的相关技术
数据均衡分布	改进的一致性哈希算法
数据备份	参数可调的弱quorum机制
数据冲突处理	向量时钟 (Vector Clock)
成员资格及错误检测	基于Gossip协议的成员资格和错误检测
5. 临时故障处理	Hinted handoff (数据回传机制)
5. 永久故障处理	Merkle哈希树

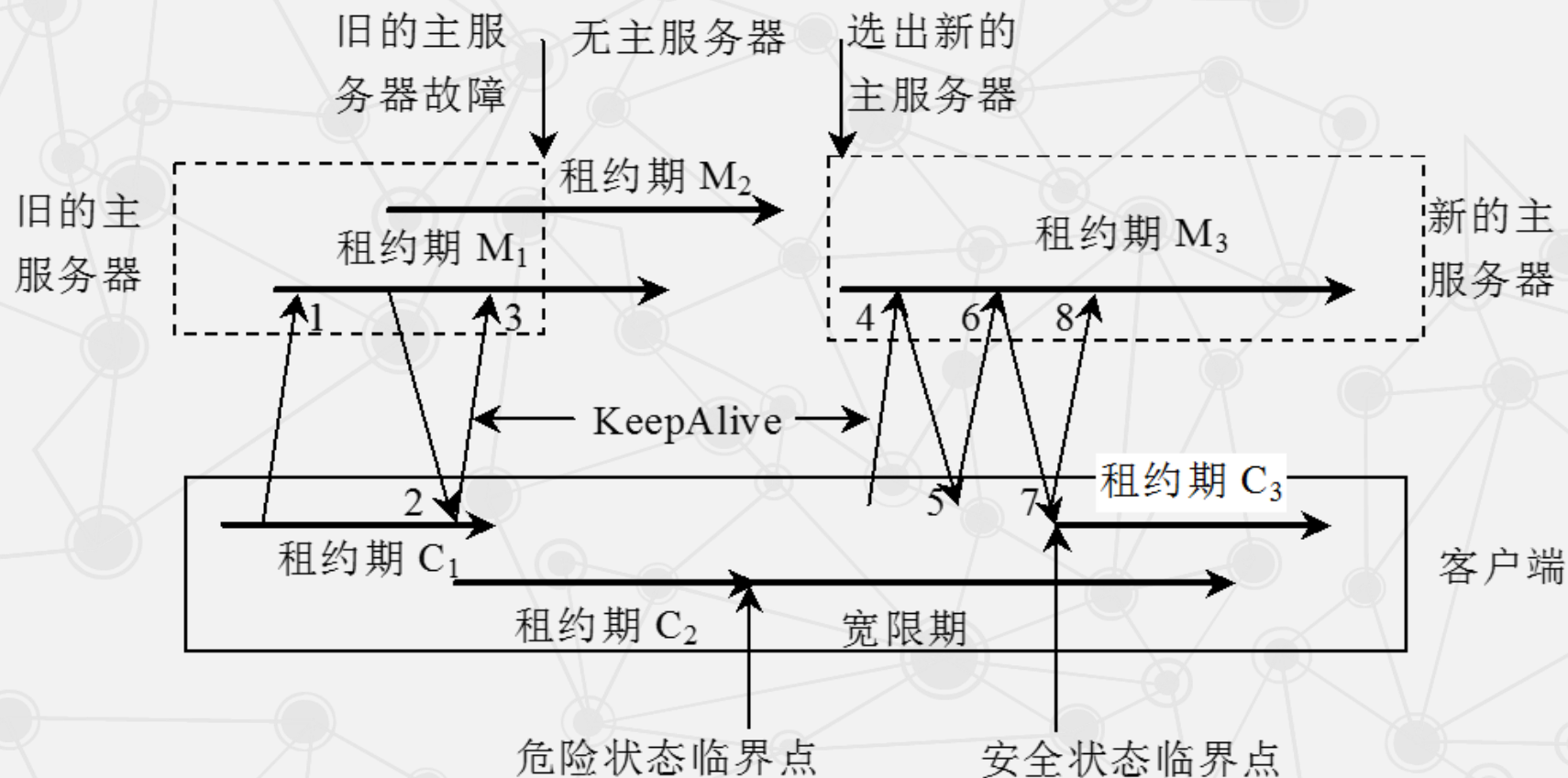
3.1 基础存储架构Dynamo

• 5. 容错机制

- Dynamo采用了**廉价**的服务器作为硬件设施，并将**物理节点失效**作为常态来处理。
- Dynamo的容错机制中包括了对**临时故障**和**永久性故障**的处理机制。

2.3 分布式锁服务Chubby

• Chubby客户端与服务器端的通信过程



客户端和主服务器之间的通信是通过KeepAlive握手协议来维持。

3.1 基础存储架构Dynamo

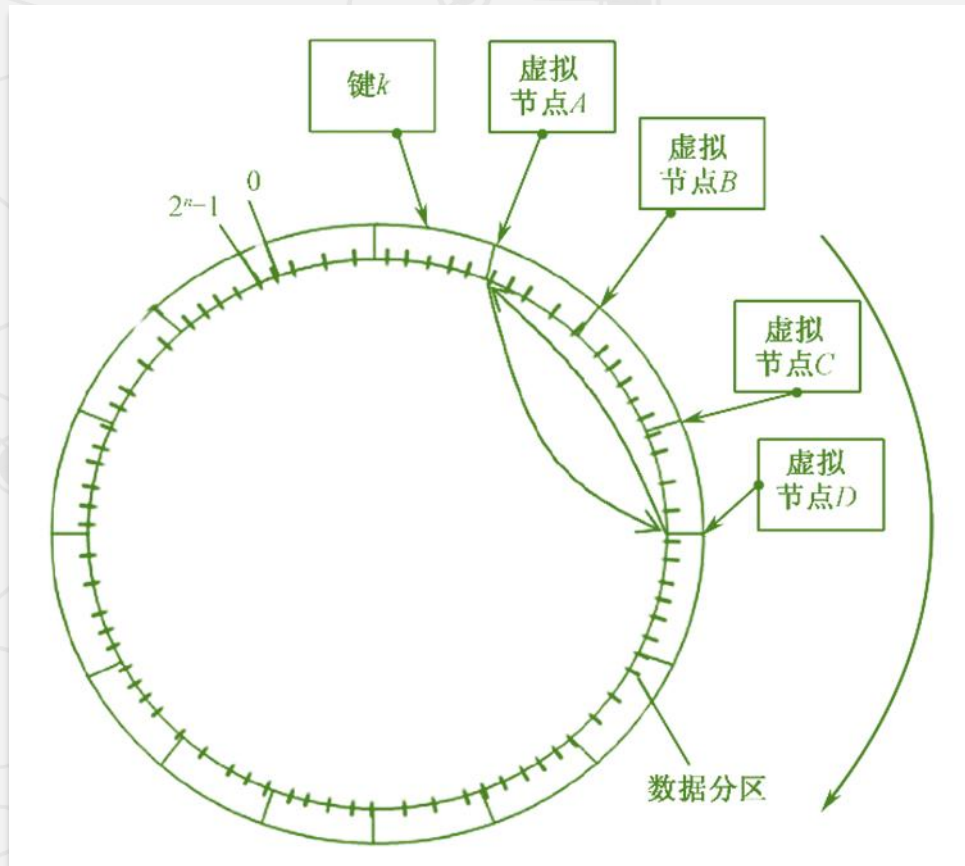
● 5. 容错机制

1) 临时故障处理机制

Dynamo中，如果某个节点由于机器假死等因素无法与其他节点通信，则会被其他节点认为失效。

这种故障是临时性的，被认为失效的节点，会在后期的闲聊（Gossip）中被发现，并重新使用。

为了处理临时失效的节点，Dynamo中采用了一种带有监听的数据回传机制（Hinted Handoff）

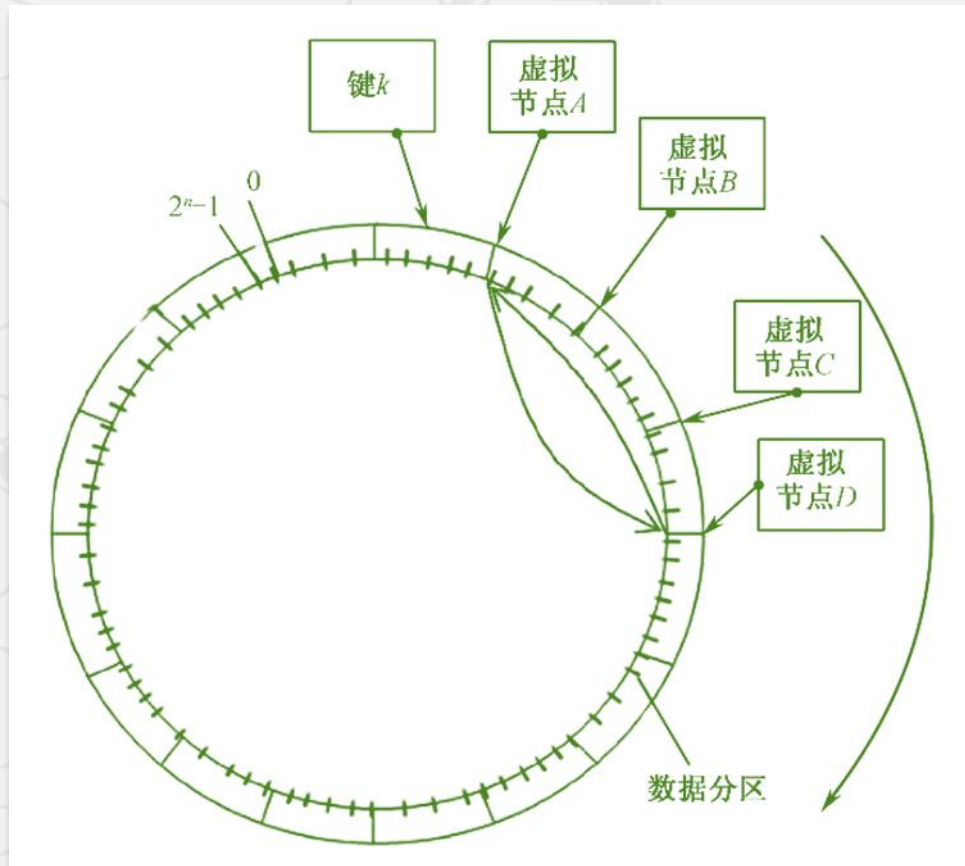


3.1 基础存储架构Dynamo

• 5. 容错机制

1) 临时故障处理机制

- 假设数据副本的数量为 N
- 当写入某个数据时，如果其 preference list 中前 N 个节点中某个节点失效
- 则会在 preference list 中第 $N+1$ 个节点上的临时空间内，记录需要在失效节点上写入的数据。

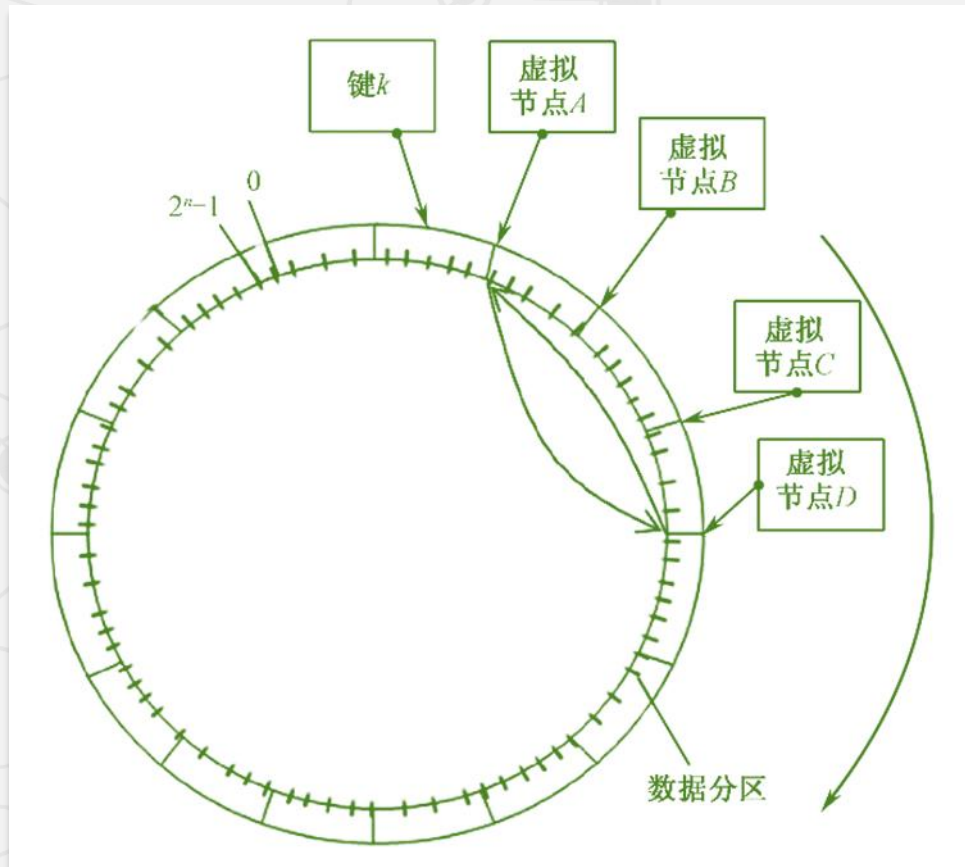


3.1 基础存储架构Dynamo

• 5. 容错机制

1) 临时故障处理机制

- 同时，第N+1个节点需要记录**失效节点的位置**，并对失效的节点进行**监测**。（即“带有**监听**”）
- 一旦失效的节点重新可用，第N+1个节点会将保存的临时数据回传给该节点，并删除临时空间中的数据。（即“**数据回传机制**”）

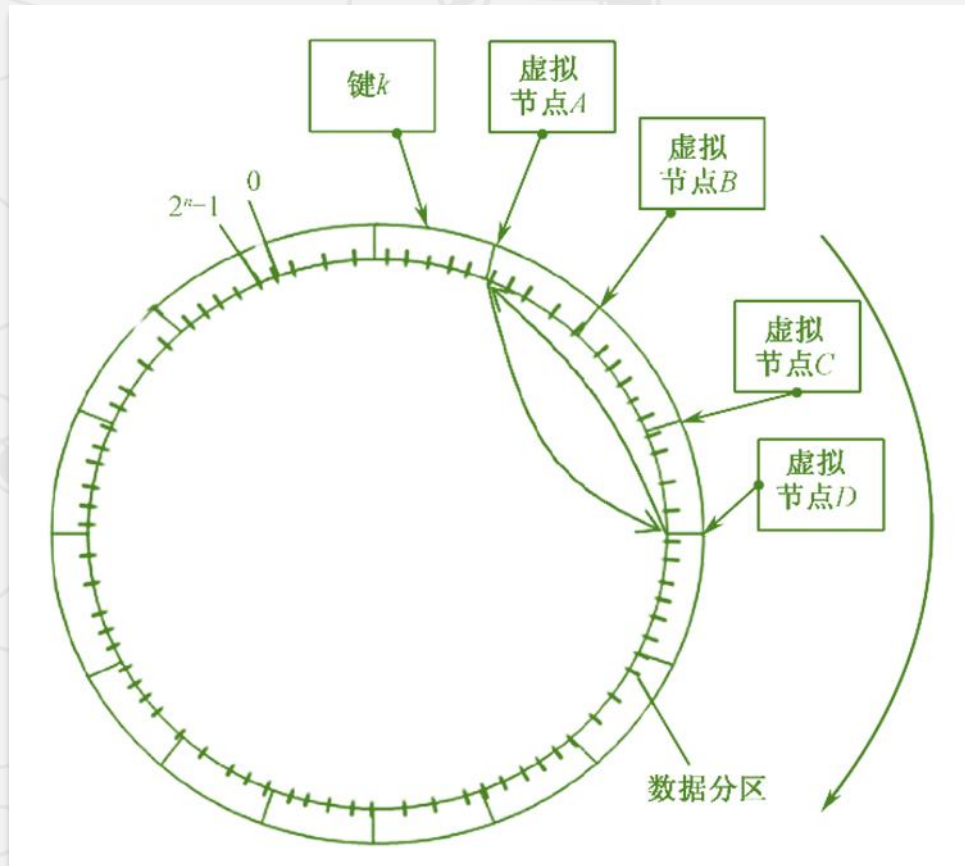


3.1 基础存储架构Dynamo

• 5. 容错机制

1) 临时故障处理机制

- 如图所示，当虚拟节点A失效后，会将数据临时存储在节点D的临时空间中，并在节点A重新可用后，由节点D将数据回传给节点A。
- 综上所述，对于临时故障的处理包括三步：临时存储->监听->回传



3.1 基础存储架构Dynamo

• 5. 容错机制

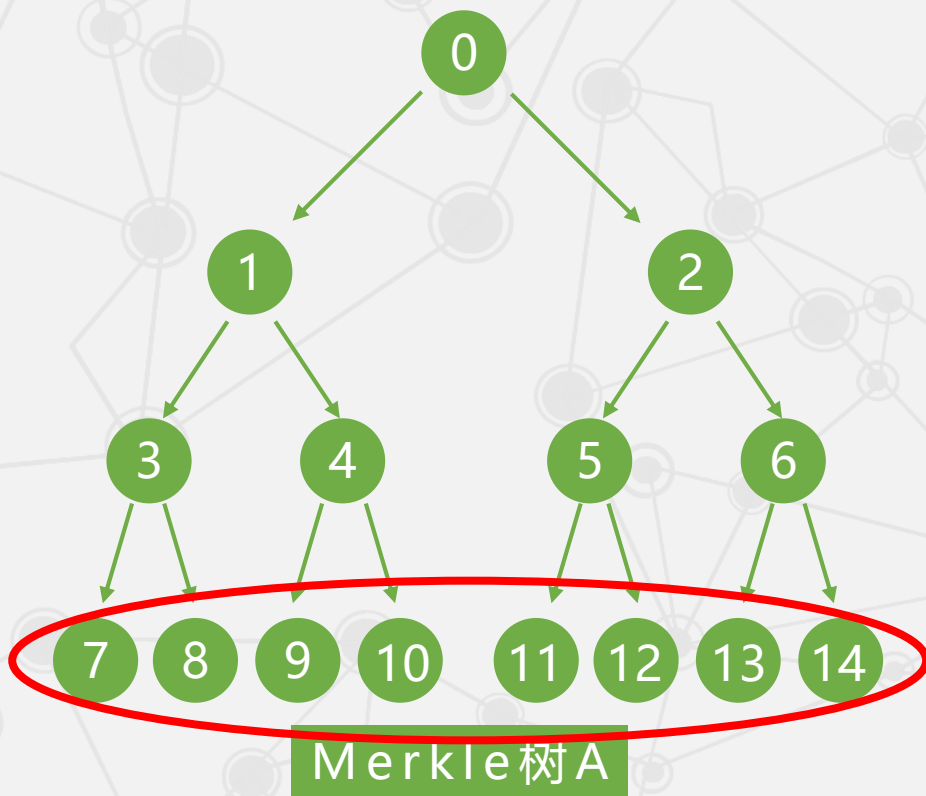
2) 永久性故障处理机制

- 在节点失效**超过了设定时间**后，如果没有发现可以重用，则Dynamo会认定该节点出现了**永久性故障**，例如，磁盘损坏等。
- 此时，Dynamo需要从其他**数据副本**进行**数据同步**。在同步过程中，为了保障数据传输的有效性，Dynamo采用**Merkle哈希树**技术来**加快检测**和**减少数据传输量**。

3.1 基础存储架构Dynamo

• 5. 容错机制

2) 永久性故障处理机制

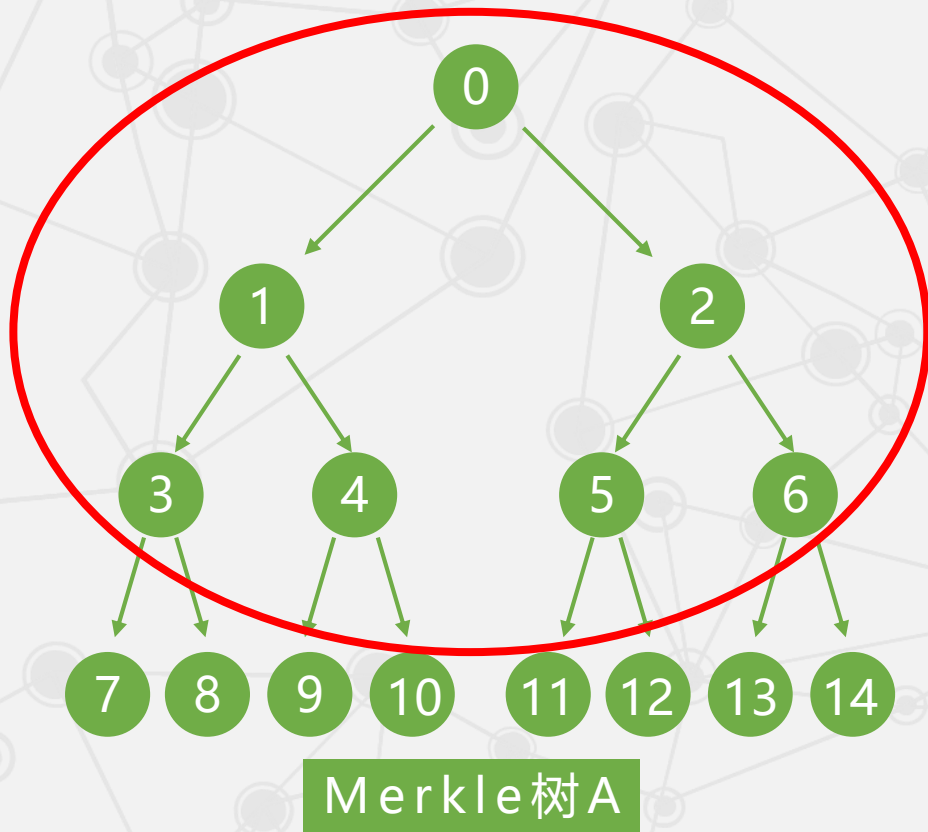


- Merkle哈希树可以为二叉树或多叉树，其中每个叶子节点的值 of 单个数据文件的哈希值

3.1 基础存储架构Dynamo

• 5. 容错机制

2) 永久性故障处理机制

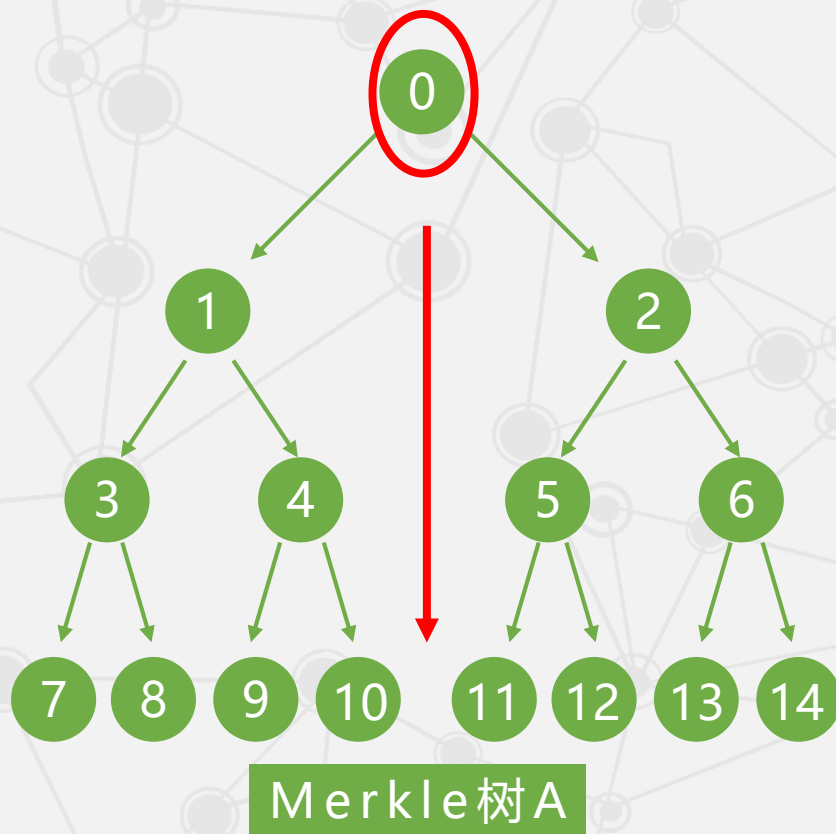


- Merkle哈希树可以为二叉树或多叉树，其中每个叶子节点的值 of 单个数据文件的哈希值
- 非叶子节点的值 of 该节点所有子节点组合后的哈希值

3.1 基础存储架构Dynamo

● 5. 容错机制

2) 永久性故障处理机制



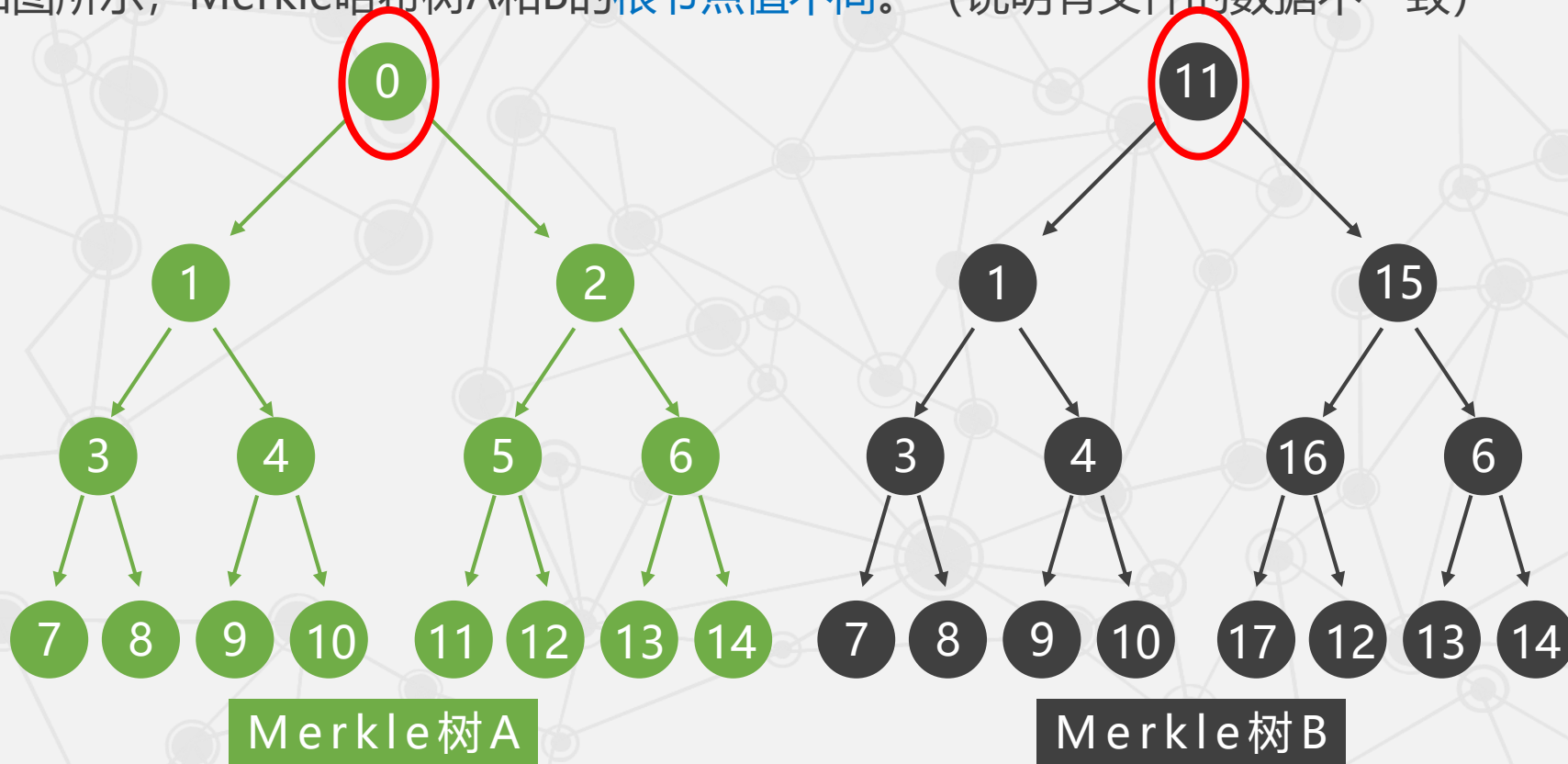
- 当采用Merkle哈希树检测数据是否一致时，系统会先比较根节点的值，如果值相同，则说明所有数据一致
- 否则需要从上往下继续比较，直到找到哈希值不同的叶子节点。

3.1 基础存储架构Dynamo

• 5. 容错机制

2) 永久性故障处理机制

如图所示，Merkle哈希树A和B的根节点值不同。（说明有文件的数据不一致）

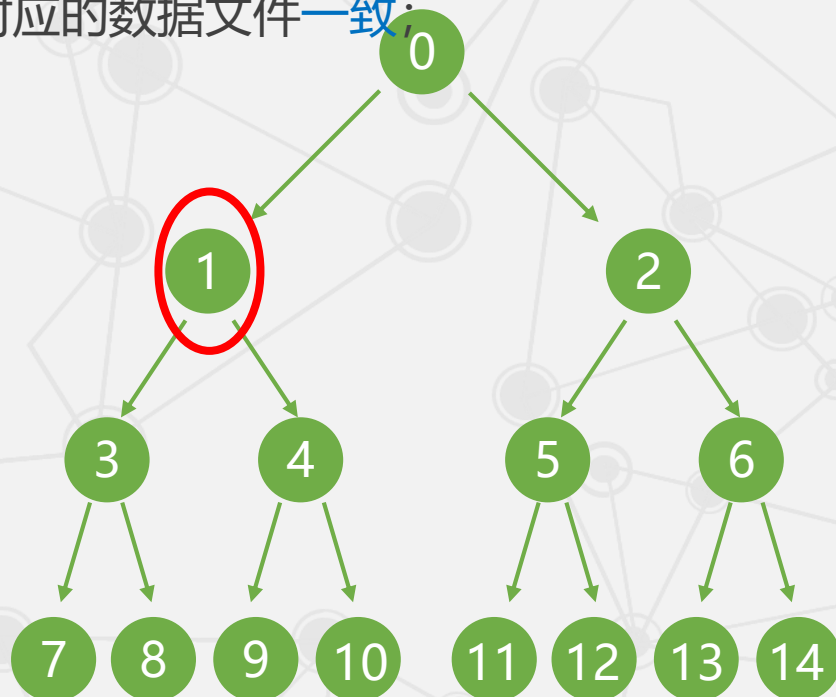


3.1 基础存储架构Dynamo

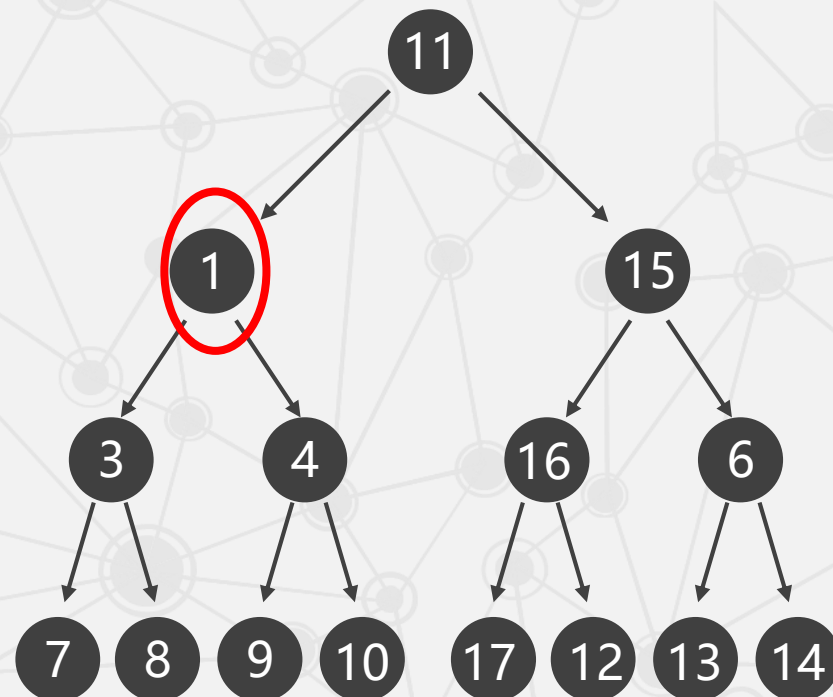
5. 容错机制

2) 永久性故障处理机制

进一步比较其子节点，发现左子节点的值相同，表明左子节点所覆盖的所有叶节点对应的数据文件一致；



Merkle 树 A



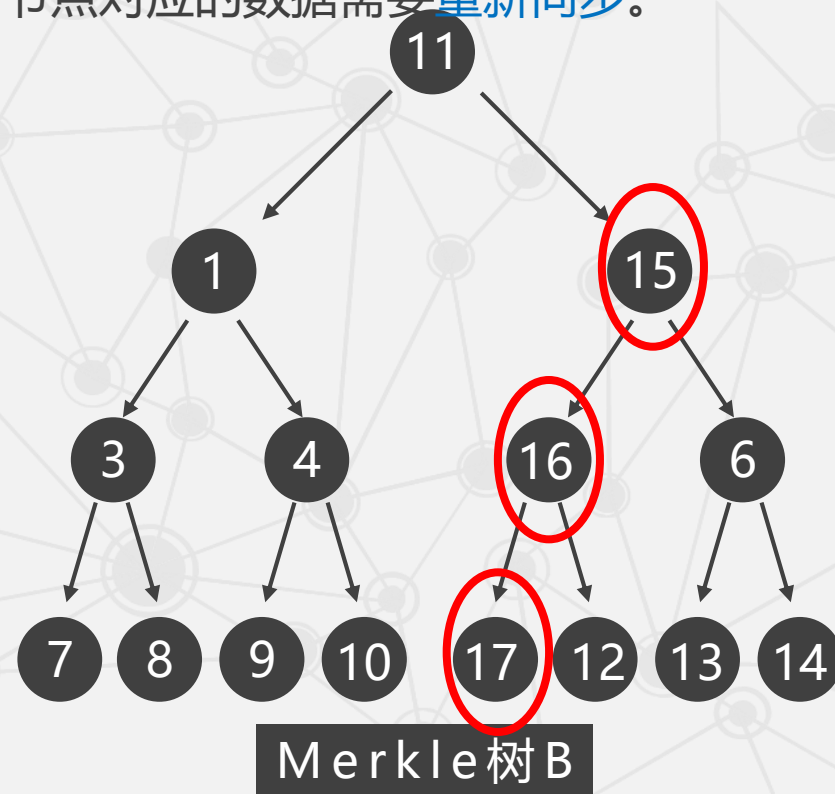
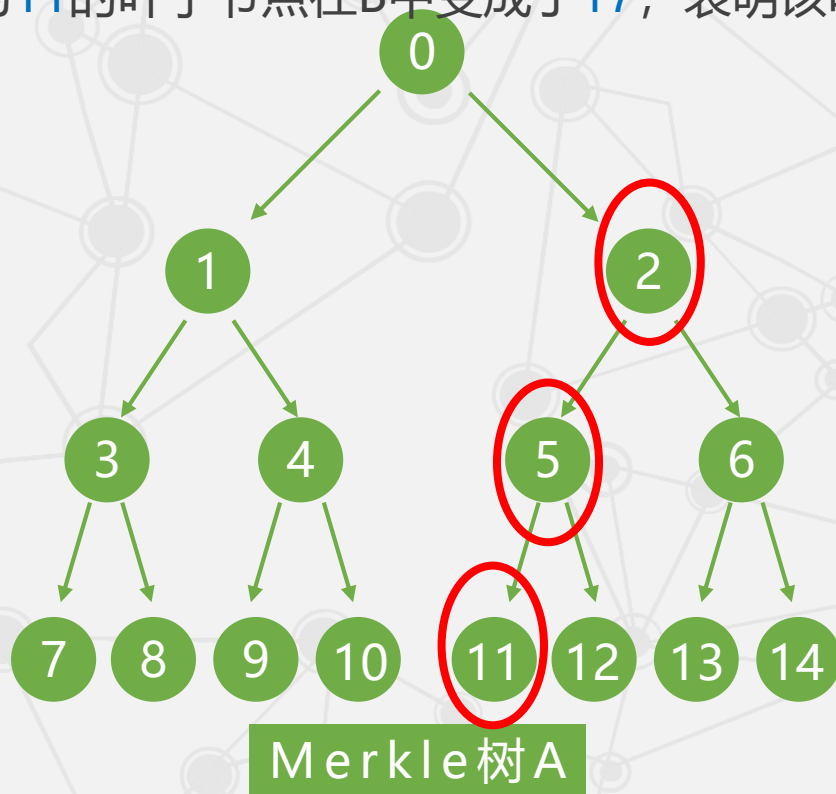
Merkle 树 B

3.1 基础存储架构Dynamo

• 5. 容错机制

2) 永久性故障处理机制

而右子节点的值不同，则进一步比较其子节点，如此下去，直至发现原来在A中值为11的叶子节点在B中变成了17，表明该叶节点对应的数据需要重新同步。

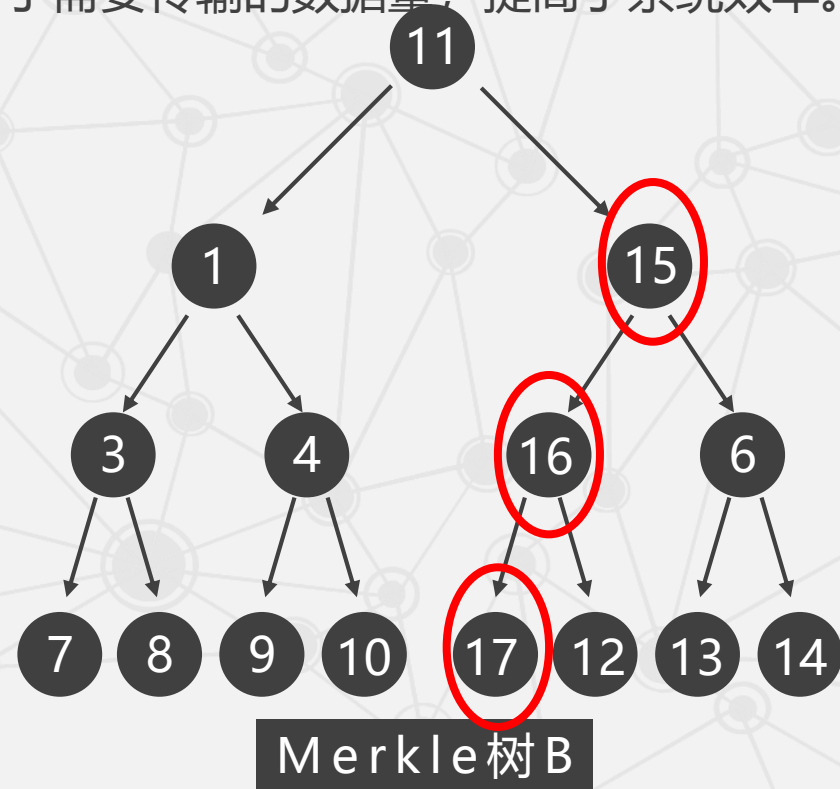
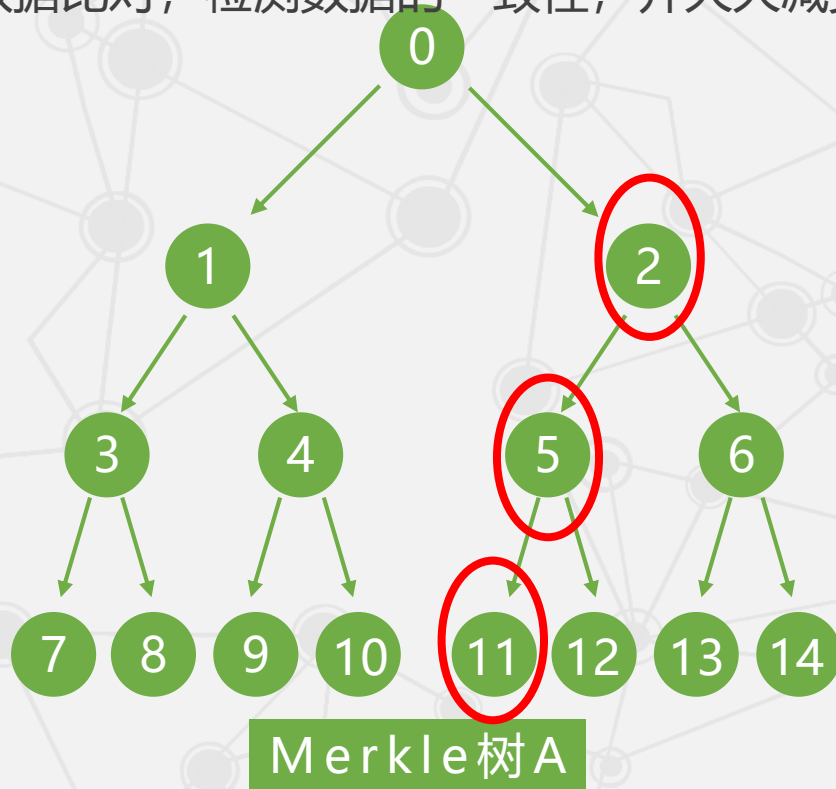


3.1 基础存储架构Dynamo

● 5. 容错机制

2) 永久性故障处理机制

通过对每个节点上每个区间的数据构建Merkle哈希树，Dynamo可以快速地进行数据比对，检测数据的一致性，并大大减少了需要传输的数据量，提高了系统效率。





本章未完待续